

AG - repetition - no_frac - Serial position analysis

```
# do this in calling R script
# library(ggplot2)
# library(fmsb) # for TestModels
# library(lme4)
# library(kableExtra)
# library(MASS) # for dropterm
# library(tidyverse)
# library(dominanceanalysis)
# library(cowplot) # for multiple plots in one figure
# library(formatters) # to wrap strings for plot titles
# library(pals) # for continuous color palettes

# load needed functions
# source(paste0(RootDir, "/src/function_library/sp_functions.R"))
# do this in the calling R script

# for testing
# CurPat <- "GM"
# CurTask <- "repetition"
# MinLength <- 4
# MaxLength <- 10
# BestModelIndexL1 <- 1
# BestModelIndexL2 <- 1
# BestModelIndexL3 <- 1
# ReportTitle <- paste0(CurPat, " - ", CurTask, " - ", RunLabel, " - Serial position analysis")
# RandomSamples <- 10
# DoSimulations <- FALSE
# RemoveFracPres <- TRUE

CurPat <- params$CurPat
CurTask <- params$CurTask
MinLength <- params$MinLength
MaxLength <- params$MaxLength
BestModelIndexL1 <- params$BestModelIndexL1
BestModelIndexL2 <- params$BestModelIndexL2
BestModelIndexL3 <- params$BestModelIndexL3
ReportTitle <- params$ReportTitle
RandomSamples <- params$RandomSamples
DoSimulations <- params$DoSimulations
RemoveFracPres <- params$RemoveFracPres

# done in calling script
# print(paste0("Using root dir: ", RootDir))
# RunDir <- paste0(paste0(RootDir, "/output/"), RunLabel))
# if(!dir.exists(RunDir)){
#   dir.create(RunDir, showWarnings = TRUE)
#   print(paste0("created run directory: ", RunDir))
# }
```

```

# # create patient directory if it doesn't already exist
# PatientDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat)
# if(!dir.exists(PatientDir)){
#   dir.create(PatientDir, showWarnings = TRUE)
#   print(paste0("created participant directory: ", PatientDir))
# }
# TablesDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/tables/")
# if(!dir.exists(TablesDir)){
#   dir.create(TablesDir, showWarnings = TRUE)
#   print(paste0("created tables directory: ", TablesDir))
# }
# FigDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/fig/")
# if(!dir.exists(FigDir)){
#   dir.create(FigDir, showWarnings = TRUE)
#   print(paste0("created figures directory: ", FigDir))
# }
# ReportsDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/reports/")
# if(!dir.exists(ReportsDir)){
#   dir.create(ReportsDir, showWarnings = TRUE)
#   print(paste0("created reports directory: ", ReportsDir))
# }
results_report_DF <- data.frame(Description=character(), ParameterValue=double())

ModelDatFilename<-paste0(RootDir, "/data/", CurPat, "/", CurPat, "_", CurTask, "_posdat.csv")
if(!file.exists(ModelDatFilename)){
  print("**ERROR** Input data file not found")
  print(sprintf("\tfile: ", ModelDatFilename))
  print(sprintf("\troot dir: ", RootDir))
}
PosDat<-read.csv(ModelDatFilename)

# may already be done in datafile
# if(RemoveFinalPosition){
#   PosDat <- PosDat[PosDat$pos < PosDat$stimlen,] # remove final position
# }
PosDat <- PosDat[PosDat$stimlen >= MinLength,] # include only lengths with sufficient N
PosDat <- PosDat[PosDat$stimlen <= MaxLength,]
# include only correct and nonword errors (not no response, word or w+nw errors)
PosDat <- PosDat[(PosDat$err_type == "correct") | (PosDat$err_type == "nonword"), ]
PosDat <- PosDat[(PosDat$pos) != (PosDat$stimlen), ]
# make sure final positions have been removed
PosDat$stim_number <- NumberStimuli(PosDat)
PosDat$raw_log_freq <- PosDat$log_freq
PosDat$log_freq <- PosDat$log_freq - mean(PosDat$log_freq) # centre frequency
OrigDataLen <- nrow(PosDat)
print(sprintf ("Original data has %d values", OrigDataLen))

## [1] "Original data has 4075 values"

if(RemoveFracPres){ # eliminate fractional preserved values?
  PosDat <- PosDat[(PosDat$preserved == 0) | (PosDat$preserved == 1),]
  DataLen<- nrow(PosDat)
  print(sprintf("Data without fractional preserved values has %d values", DataLen))
  print(sprintf("%d values eliminated.", OrigDataLen - DataLen))
}

```

```
## [1] "Data without fractional preserved values has 4022 values"
## [1] "53 values eliminated."
```

```
PosDat$CumPres <- CalcCumPres(PosDat)
PosDat$CumErr <- CalcCumErrFromPreserved(PosDat)
```

```
# plot distribution of complex onsets and codas
# across serial position in the stimuli -- do complexities concentrate
# on one part of the serial position curve?
# syll_component = 1 is coda
# syll_component = S is satellite
```

```
# get counts of syllable components in different serial positions
```

```
syll_comp_dist_N <- PosDat %>% mutate(pos_factor = as.factor(pos), syll_component_factor = as.factor(syll_component))
```

```
syll_comp_dist_perc <- syll_comp_dist_N %>% ungroup() %>%
  mutate(across(O:S, ~ . / total))
```

```
kable(syll_comp_dist_N)
```

pos_factor	O	P	V	1	S	total
1	506	29	119	NA	NA	654
2	58	NA	399	90	104	651
3	287	NA	156	193	13	649
4	279	NA	217	62	37	595
5	214	NA	201	65	37	517
6	197	1	127	63	22	410
7	164	NA	100	25	17	306
8	86	NA	47	26	4	163
9	69	NA	2	NA	6	77

```
kable(syll_comp_dist_perc)
```

pos_factor	O	P	V	1	S	total
1	0.7737003	0.0443425	0.1819572	NA	NA	654
2	0.0890937	NA	0.6129032	0.1382488	0.1597542	651
3	0.4422188	NA	0.2403698	0.2973806	0.0200308	649
4	0.4689076	NA	0.3647059	0.1042017	0.0621849	595
5	0.4139265	NA	0.3887814	0.1257253	0.0715667	517
6	0.4804878	0.0024390	0.3097561	0.1536585	0.0536585	410
7	0.5359477	NA	0.3267974	0.0816993	0.0555556	306
8	0.5276074	NA	0.2883436	0.1595092	0.0245399	163
9	0.8961039	NA	0.0259740	NA	0.0779221	77

```
# get percentages only from summary table
```

```
syll_comp_perc <- syll_comp_dist_perc[,2:(ncol(syll_comp_dist_perc)-1)]
```

```
syll_comp_perc$pos <- as.integer(as.character(syll_comp_dist_perc$pos_factor))
```

```
syll_comp_perc <- syll_comp_perc %>% rename("Coda" = "1", "Satellite" = "S")
```

```
# pivot to long format for plotting
```

```
syll_comp_plot_data <- syll_comp_perc %>% pivot_longer(cols=seq(1:(ncol(syll_comp_perc)-1)),
  names_to="syll_component", values_to="percent") %>% filter(syll_component %in% c("Coda", "Satellite"))
```

```
# plot satellite and coda occurrences across position
```

```
syll_comp_plot <- ggplot(syll_comp_plot_data,
  aes(x=pos,y=percent,group=syll_component,
    color=syll_component,
    linetype = syll_component,
    shape = syll_component)) +
  geom_line() + geom_point() + scale_color_manual(name="Syllable component", values = palette_values) +
syll_comp_plot <- syll_comp_plot + scale_y_continuous(name="Percent of segment types") + scale_linetype
  scale_shape_discrete(name="Syllable component")
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask, "_pos_of_syll_complexities_in_stimuli.png"), plot=s
```

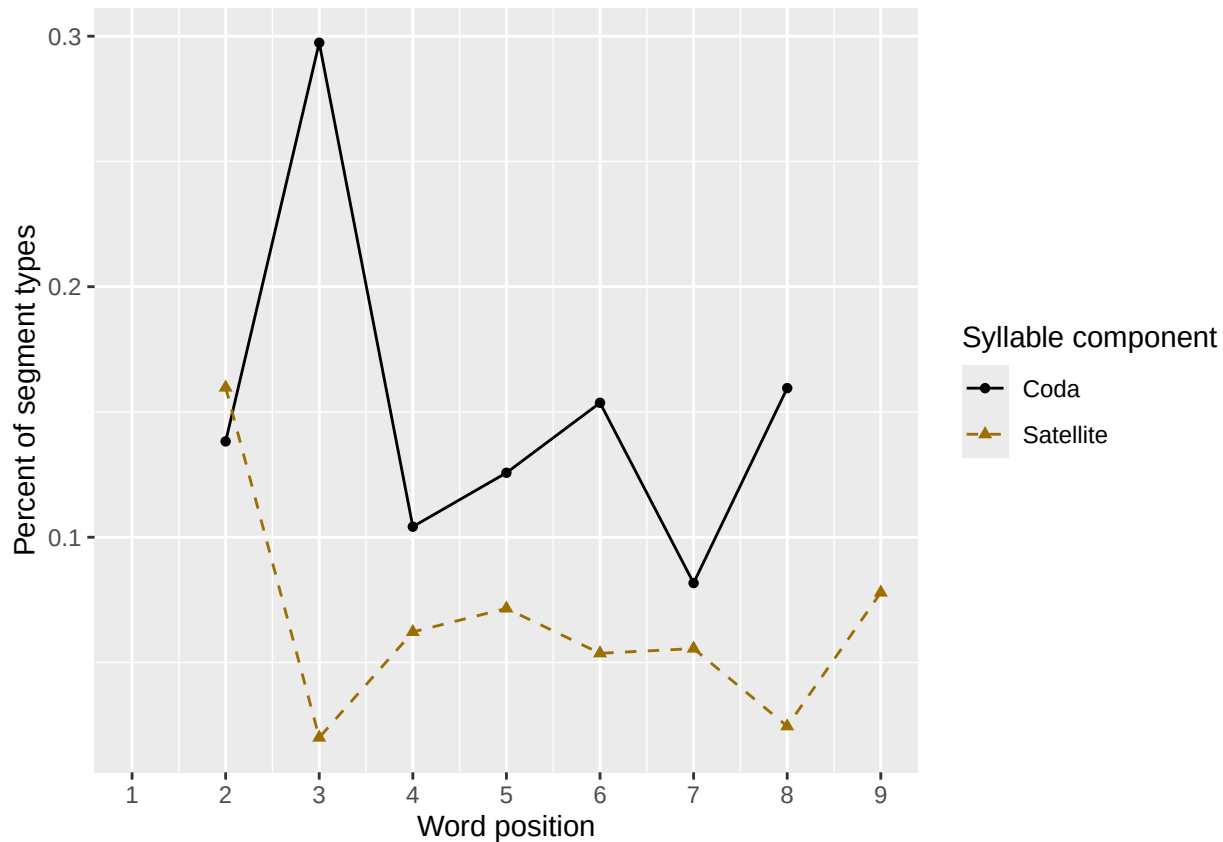
```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_line()`).
```

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_point()`).
```

```
syll_comp_plot
```

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_line()`).
```

```
## Removed 3 rows containing missing values or values outside the scale range (`geom_point()`).
```



```
# len/pos table
```

```
pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(preserved = mean(preserved))
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
min_preserved_raw <- min(as.numeric(unlist(pos_len_summary$preserved)))
```

```
max_preserved_raw <- max(as.numeric(unlist(pos_len_summary$preserved)))
```

```
preserved_range <- (max_preserved_raw - min_preserved_raw)
```

```

min_preserved <- min_preserved_raw - (0.1 * preserved_range)
max_preserved <- max_preserved_raw + (0.1 * preserved_range)
pos_len_table <- pos_len_summary %>% pivot_wider(names_from = pos, values_from = preserved)
write.csv(pos_len_table, paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask, "_preserved_percentages_by_len",
pos_len_table

```

```

## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1      4 0.961 0.941 1      NA      NA      NA      NA      NA      NA
## 2      5 0.916 0.940 0.855 0.916 NA      NA      NA      NA      NA
## 3      6 0.933 0.942 0.914 0.924 0.876 NA      NA      NA      NA
## 4      7 0.927 0.954 0.935 0.953 0.954 0.862 NA      NA      NA
## 5      8 0.923 0.972 0.950 0.921 0.880 0.936 0.945 NA      NA
## 6      9 0.895 0.943 0.897 0.953 0.906 0.906 0.872 0.874 NA
## 7     10 0.922 0.987 0.88   0.88   0.934 0.947 0.92   0.908 0.922

```

```

# len/pos table
pos_len_N <- PosDat %>% group_by(stimlen, pos) %>% summarise(preserved = mean(preserved), N=n()) %>% dplyr::

```

`summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

```

pos_len_N_table <- pos_len_N %>% pivot_wider(names_from = pos, values_from = N)
write.csv(pos_len_N_table, paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
                                "_N_by_len_position.csv"))
pos_len_N_table

```

```

## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <int> <int> <int> <int> <int> <int> <int> <int> <int>
## 1      4    51    51    51    NA    NA    NA    NA    NA    NA
## 2      5    83    83    83    83    NA    NA    NA    NA    NA
## 3      6   105   104   105   105   105    NA    NA    NA    NA
## 4      7   109   109   107   107   109   109    NA    NA    NA
## 5      8   143   142   141   139   142   141   145    NA    NA
## 6      9    86    87    87    86    85    85    86    87    NA
## 7     10    77    75    75    75    76    75    75    76    77

```

```

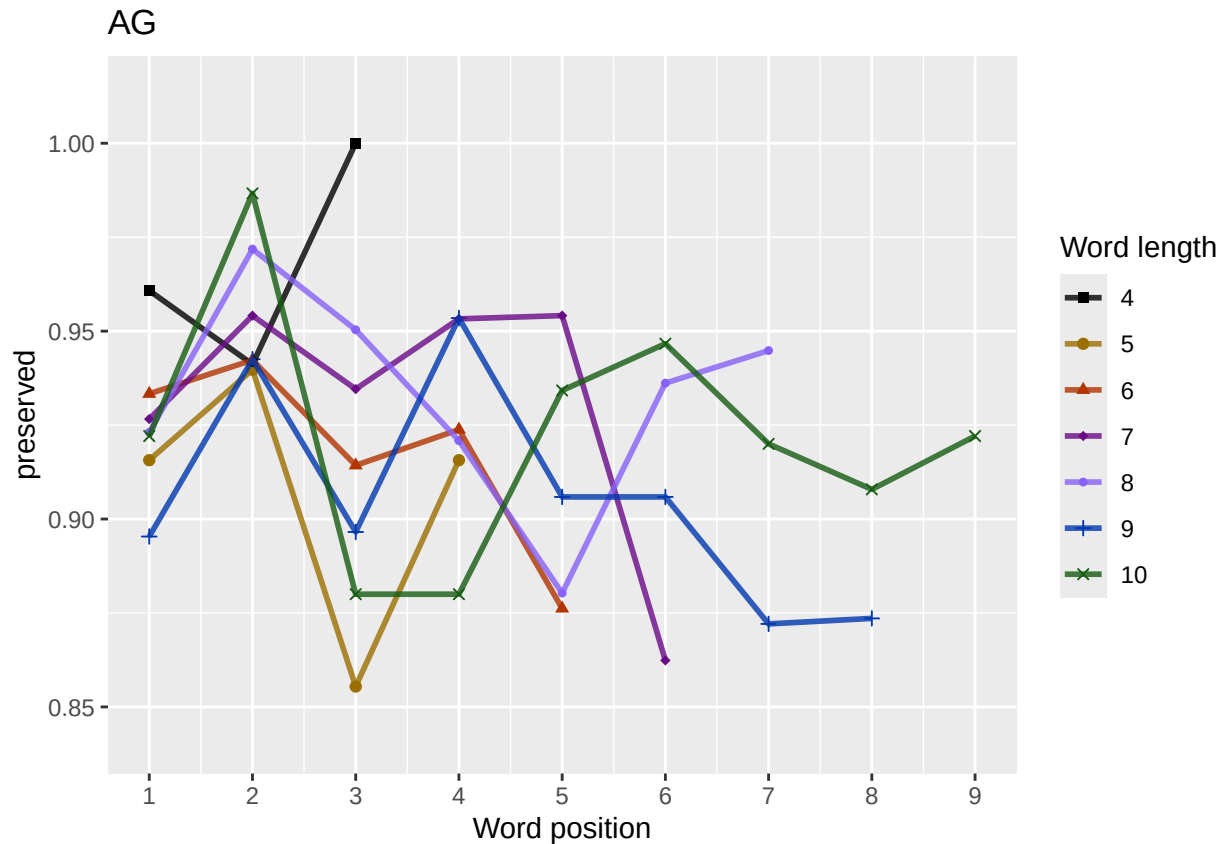
obs_linetypes <- c("solid", "solid", "solid", "solid",
                  "solid", "solid", "solid", "solid", "solid")
pred_linetypes <- "dashed"
pos_len_summary$stimlen <- factor(pos_len_summary$stimlen)
pos_len_summary$pos <- factor(pos_len_summary$pos)
# len_pos_plot <- ggplot(pos_len_summary, aes(x=pos, y=preserved, group=stimlen, shape=stimlen, color=stimlen))
len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
                                           paste0(CurPat),
                                           NULL,
                                           c(min_preserved, max_preserved),
                                           palette_values,
                                           shape_values,
                                           obs_linetypes,
                                           pred_linetypes)

```

`summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
```

```
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask,
              "_percent_preserved_by_length_pos.png"),
       plot=len_pos_plot, device="png", unit="cm", width=15, height=11)
len_pos_plot
```



Length and position

```
# length and position
```

```
LPMModelEquations<-c("preserved ~ 1",
                      "preserved ~ stimlen",
                      "preserved ~ pos",
                      "preserved ~ stimlen + pos",
                      "preserved ~ stimlen*pos",
                      "preserved ~ I(pos^2)+pos",
                      "preserved ~ stimlen + I(pos^2) + pos",
                      "preserved ~ stimlen * (I(pos^2) + pos)"
)
```

```
LPres<-TestModels(LPMModelEquations,PosDat)
```

```
## *****
```

```
## model index: 3
```

```
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
```

```
## data = PosDat)
```

```

##
## Coefficients:
## (Intercept)          pos
##      2.73419      -0.06247
##
## Degrees of Freedom: 4021 Total (i.e. Null);  4020 Residual
## Null Deviance:      2180
## Residual Deviance: 2174  AIC: 2178
## log likelihood:  -1087.135
## Nagelkerke R2:  0.003126797
## % pres/err predicted correctly:  -568.7646
## % of predictable range [ (model-null)/(1-null) ]:  0.001325101
## *****
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##      2.770121      0.002423      -0.083995
##
## Degrees of Freedom: 4021 Total (i.e. Null);  4019 Residual
## Null Deviance:      2180
## Residual Deviance: 2174  AIC: 2180
## log likelihood:  -1087.115
## Nagelkerke R2:  0.003150882
## % pres/err predicted correctly:  -568.7586
## % of predictable range [ (model-null)/(1-null) ]:  0.001335752
## *****
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##      2.708331      0.004004      -0.063750
##
## Degrees of Freedom: 4021 Total (i.e. Null);  4019 Residual
## Null Deviance:      2180
## Residual Deviance: 2174  AIC: 2180
## log likelihood:  -1087.13
## Nagelkerke R2:  0.003132907
## % pres/err predicted correctly:  -568.7622
## % of predictable range [ (model-null)/(1-null) ]:  0.001329402
## *****
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos  stimlen:pos

```

```

##      3.21330      -0.05742      -0.21998      0.01840
##
## Degrees of Freedom: 4021 Total (i.e. Null);  4018 Residual
## Null Deviance:      2180
## Residual Deviance: 2173  AIC: 2181
## log likelihood:  -1086.657
## Nagelkerke R2:  0.003694015
## % pres/err predicted correctly:  -568.6275
## % of predictable range [ (model-null)/(1-null) ]:  0.001565498
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)
##      2.486
##
## Degrees of Freedom: 4021 Total (i.e. Null);  4021 Residual
## Null Deviance:      2180
## Residual Deviance: 2180  AIC: 2182
## log likelihood:  -1089.768
## Nagelkerke R2:  2.653751e-16
## % pres/err predicted correctly:  -569.5206
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****
## model index:  7
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)      pos
##      2.750672      0.002680      0.002279      -0.083564
##
## Degrees of Freedom: 4021 Total (i.e. Null);  4018 Residual
## Null Deviance:      2180
## Residual Deviance: 2174  AIC: 2182
## log likelihood:  -1087.113
## Nagelkerke R2:  0.003153534
## % pres/err predicted correctly:  -568.7572
## % of predictable range [ (model-null)/(1-null) ]:  0.001338067
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      2.71570      -0.02971
##
## Degrees of Freedom: 4021 Total (i.e. Null);  4020 Residual

```



```
## Null Deviance:      2180
## Residual Deviance: 2179 AIC: 2183
## log likelihood: -1089.421
## Nagelkerke R2: 0.0004123246
## % pres/err predicted correctly: -569.4232
## % of predictable range [ (model-null)/(1-null) ]: 0.0001708448
## *****
## model index: 8
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)      pos  stimlen:I(pos^2)      stimlen:I(pos^2)
## 2.864530      -0.030138      -0.046046      0.066079      0.004261      -0.004261
##
## Degrees of Freedom: 4021 Total (i.e. Null); 4016 Residual
## Null Deviance:      2180
## Residual Deviance: 2173 AIC: 2185
## log likelihood: -1086.404
## Nagelkerke R2: 0.00399498
## % pres/err predicted correctly: -568.554
## % of predictable range [ (model-null)/(1-null) ]: 0.00169423
## *****
```

```
BestLPModel<-LPRes$ModelResult[[1]]
BestLPModelFormula<-LPRes$Model[[1]]

LPAICSummary<-data.frame(Model=LPRes$Model,
                          AIC=LPRes$AIC,
                          row.names = LPRes$Model)
LPAICSummary$DeltaAIC<-LPAICSummary$AIC-LPAICSummary$AIC[1]
LPAICSummary$AICexp<-exp(-0.5*LPAICSummary$DeltaAIC)
LPAICSummary$AICwt<-LPAICSummary$AICexp/sum(LPAICSummary$AICexp)
LPAICSummary$NagR2<-LPRes$NagR2

LPAICSummary <- merge(LPAICSummary,LPRes$CoefficientValues, by='row.names',sort=FALSE)
LPAICSummary <- subset(LPAICSummary, select = -c(Row.names))

write.csv(LPAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                              "_len_pos_model_summary.csv"),row.names = FALSE)

kable(LPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos^2)	stimlen:I(pos^2)	stimlen:I(pos^2)
preserved ~ pos	2178.271	0.000000	0.000000	0.000000	0.000000	2.864530	-0.030138	NA	-	NA	NA
								0.0624749			
preserved ~ I(pos^2) + pos	2180.230	0.959410	0.375420	0.115404	0.000315	2.864530	-0.030138	NA	-	NA	0.0024234
								0.0839953			
preserved ~ stimlen + pos	2180.260	0.989705	0.369778	0.115172	0.000313	2.864530	-0.030138	0.0040043	-	NA	NA
								0.0637498			
preserved ~ stimlen * pos	2181.313	3.044210	0.218250	0.098955	0.000369	2.864530	-0.030138	-	-	0.0184031	NA
								0.0574240		0.2199820	
preserved ~ 1	2181.533	3.264728	0.195466	0.098020	0.000000	2.864530	-0.030138	NA	NA	NA	NA

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:pos	I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen + I(pos^2) + pos	2182.226	0.954952	1.138418	0.2056795	0.003153	257506720.0026796	-	NA	0.0022789	NA	0.0835644
preserved ~ stimlen	2182.844	1.570873	1.101729	0.004174	0.0000412	3715702	-	NA	NA	NA	NA
preserved ~ stimlen * (I(pos^2) + pos)	2184.808	0.536978	0.038063	0.015618	0.003992	20864530	-	0.0660793	-	-	0.0042605
							0.0301381		0.0061340	0.0460457	

```
print(BestLPModelFormula)
```

```
## [1] "preserved ~ pos"
```

```
print(BestLPModel)
```

```
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      2.73419      -0.06247
##
## Degrees of Freedom: 4021 Total (i.e. Null);  4020 Residual
## Null Deviance:      2180
## Residual Deviance: 2174  AIC: 2178
```

```
PosDat$LPFitted<-fitted(BestLPModel)
fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat, PosDat$patient[1],
  NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
```

```
fitted_pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPfitted))
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
fitted_pos_len_table <- fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_from = fitted)
fitted_pos_len_table
```

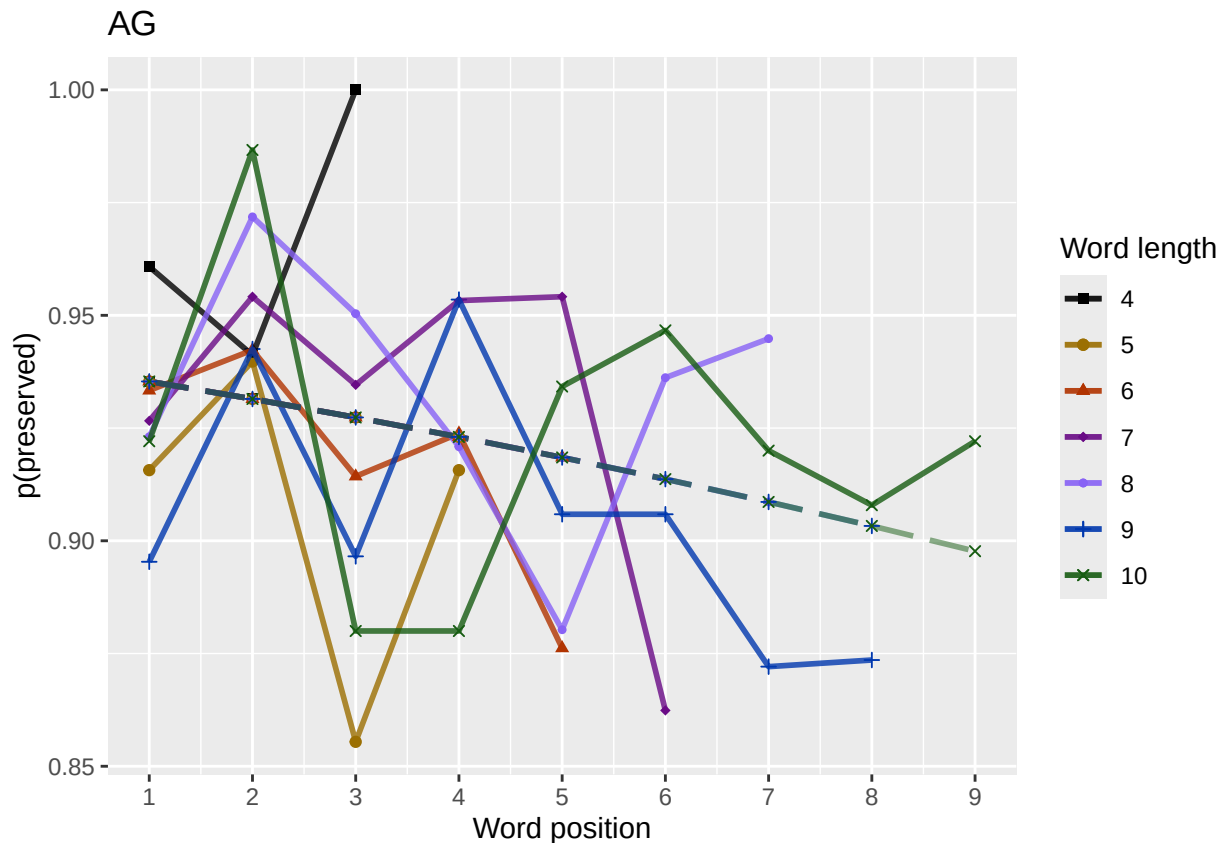
```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1     4 0.935 0.931 0.927 NA      NA      NA      NA      NA      NA
## 2     5 0.935 0.931 0.927 0.923 NA      NA      NA      NA      NA
## 3     6 0.935 0.931 0.927 0.923 0.918 NA      NA      NA      NA
## 4     7 0.935 0.931 0.927 0.923 0.918 0.914 NA      NA      NA
## 5     8 0.935 0.931 0.927 0.923 0.918 0.914 0.909 NA      NA
## 6     9 0.935 0.931 0.927 0.923 0.918 0.914 0.909 0.903 NA
## 7    10 0.935 0.931 0.927 0.923 0.918 0.914 0.909 0.903 0.898
```

```

fitted_pos_len_summary$stimlen<-factor(fitted_pos_len_summary$stimlen)
fitted_pos_len_summary$pos<-factor(fitted_pos_len_summary$pos)
# fitted_len_pos_plot <- ggplot(pos_len_summary, aes(x=pos, y=preserved, group=stimlen, shape=stimlen, color=stimlen))
# fitted_len_pos_plot <- fitted_len_pos_plot + geom_line(data=fitted_pos_len_summary, aes(x=pos, y=fitted, group=stimlen, shape=stimlen, color=stimlen))
# geom_point(data=fitted_pos_len_summary, aes(x=pos, y=fitted, group=stimlen, shape=stimlen, color=stimlen)) + ggtitle(paste0(
fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
  paste0(PosDat$patient[1]),
  "LPFitted",
  NULL,
  palette_values,
  shape_values,
  obs_linetypes,
  pred_linetypes = c("longdash")
)

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask, "_percent_preserved_by_length_pos_wfit.png"), plot=fitted_len_pos_plot

```



length and position without fragments to see if this changes position² influence

```

# first number responses, then count resp with fragments - below we will eliminate fragments
# and re-run models

```

```

# number responses
resp_num<-0
prev_pos<-9999 # big number to initialize (so first position is smaller)
resp_num_array <- integer(length = nrow(PosDat))
for(i in seq(1,nrow(PosDat))){
  if(PosDat$pos[i] <= prev_pos){ # equal for resp length 1
    resp_num <- resp_num + 1
  }
  resp_num_array[i]<-resp_num
  prev_pos <- PosDat$pos[i]
}
PosDat$resp_num <- resp_num_array

# count responses with fragments
resp_with_frag <- PosDat %>% group_by(resp_num) %>% summarise(frag = as.logical(sum(fragment)))
num_frag <- resp_with_frag %>% summarise(frag_sum = sum(frag), N=n())
num_frag

## # A tibble: 1 x 2
##   frag_sum      N
##   <int> <int>
## 1      11  657

num_frag$percent_with_frag[1] <- (num_frag$frag_sum[1] / num_frag$N[1])*100

## Warning: Unknown or uninitialised column: `percent_with_frag`.

print(sprintf("The number of responses with fragments was %d / %d = %.2f percent",
  num_frag$frag_sum[1],num_frag$N[1],num_frag$percent_with_frag[1]))

## [1] "The number of responses with fragments was 11 / 657 = 1.67 percent"

write.csv(num_frag,paste0(TablesDir,CurPat,"_",RunLabel,"_",
  CurTask,"_percent_fragments.csv"),row.names = FALSE)

# length and position models for data without fragments
NoFragData <- PosDat %>% filter(fragment != 1)

NoFrag_LPRes<-TestModels(LPModelEquations,NoFragData)

## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##   data = PosDat)
##
## Coefficients:
## (Intercept)
##      2.564
##
## Degrees of Freedom: 3998 Total (i.e. Null); 3998 Residual
## Null Deviance: 2060
## Residual Deviance: 2060 AIC: 2062
## log likelihood: -1029.933
## Nagelkerke R2: -5.515864e-16
## % pres/err predicted correctly: -530.0918
## % of predictable range [ (model-null)/(1-null) ]: 0

```

```

## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          pos
## 2.67986      -0.02987
##
## Degrees of Freedom: 3998 Total (i.e. Null); 3997 Residual
## Null Deviance: 2060
## Residual Deviance: 2059 AIC: 2063
## log likelihood: -1029.382
## Nagelkerke R2: 0.0006847865
## % pres/err predicted correctly: -529.9448
## % of predictable range [ (model-null)/(1-null) ]: 0.0002767747
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
## 2.75860      -0.02527
##
## Degrees of Freedom: 3998 Total (i.e. Null); 3997 Residual
## Null Deviance: 2060
## Residual Deviance: 2059 AIC: 2063
## log likelihood: -1029.698
## Nagelkerke R2: 0.0002914502
## % pres/err predicted correctly: -530.0292
## % of predictable range [ (model-null)/(1-null) ]: 0.0001178813
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
## 2.75736      -0.01189      -0.02620
##
## Degrees of Freedom: 3998 Total (i.e. Null); 3996 Residual
## Null Deviance: 2060
## Residual Deviance: 2059 AIC: 2065
## log likelihood: -1029.338
## Nagelkerke R2: 0.0007387883
## % pres/err predicted correctly: -529.9337
## % of predictable range [ (model-null)/(1-null) ]: 0.0002975913
## *****
## model index: 6
##

```

```

## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##    2.708587    0.002017   -0.047495
##
## Degrees of Freedom: 3998 Total (i.e. Null);  3996 Residual
## Null Deviance:      2060
## Residual Deviance: 2059  AIC: 2065
## log likelihood:  -1029.369
## Nagelkerke R2:   0.0007006445
## % pres/err predicted correctly:  -529.941
## % of predictable range [ (model-null)/(1-null) ]:  0.0002838042
## *****
## model index:  5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos  stimlen:pos
##    2.988610   -0.040011   -0.100042    0.008695
##
## Degrees of Freedom: 3998 Total (i.e. Null);  3995 Residual
## Null Deviance:      2060
## Residual Deviance: 2058  AIC: 2066
## log likelihood:  -1029.241
## Nagelkerke R2:   0.000860237
## % pres/err predicted correctly:  -529.91
## % of predictable range [ (model-null)/(1-null) ]:  0.0003423486
## *****
## model index:  7
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos
##    2.806313   -0.013408    0.002743   -0.049706
##
## Degrees of Freedom: 3998 Total (i.e. Null);  3995 Residual
## Null Deviance:      2060
## Residual Deviance: 2059  AIC: 2067
## log likelihood:  -1029.315
## Nagelkerke R2:   0.0007672549
## % pres/err predicted correctly:  -529.9274
## % of predictable range [ (model-null)/(1-null) ]:  0.0003094234
## *****
## model index:  8
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##

```

```
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)      pos  stimlen:I(pos^2)      stimlen:I(pos^2)
##      3.134661      -0.058976      0.012445      -0.197538      -0.001519      0.0003511845
##
## Degrees of Freedom: 3998 Total (i.e. Null);  3993 Residual
## Null Deviance:      2060
## Residual Deviance: 2058  AIC: 2070
## log likelihood:  -1029.225
## Nagelkerke R2:  0.0008799916
## % pres/err predicted correctly:  -529.9053
## % of predictable range [ (model-null)/(1-null) ]:  0.0003511845
## *****
```

```
NoFragBestLPModel<-NoFrag_LPres$ModelResult[[1]]
NoFragBestLPModelFormula<-NoFrag_LPres$Model[[1]]

NoFragLPAICSummary<-data.frame(Model=NoFrag_LPres$Model,
                                AIC=NoFrag_LPres$AIC,
                                row.names = NoFrag_LPres$Model)
NoFragLPAICSummary$DeltaAIC<-NoFragLPAICSummary$AIC-NoFragLPAICSummary$AIC[1]
NoFragLPAICSummary$AICexp<-exp(-0.5*NoFragLPAICSummary$DeltaAIC)
NoFragLPAICSummary$AICwt<-NoFragLPAICSummary$AICexp/sum(NoFragLPAICSummary$AICexp)
NoFragLPAICSummary$NagR2<-NoFrag_LPres$NagR2

NoFragLPAICSummary <- merge(NoFragLPAICSummary,NoFrag_LPres$CoefficientValues, by='row.names',sort=FALSE)
NoFragLPAICSummary <- subset(NoFragLPAICSummary, select = -c(Row.names))

write.csv(NoFragLPAICSummary,paste0(TablesDir,
                                    CurPat,"_",RunLabel,"_",
                                    CurTask,
                                    "_no_fragments_len_pos_model_summary.csv"),
          row.names = FALSE)
kable(NoFragLPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos^2)	stimlen:I(pos^2)
preserved ~ 1	2061.866	0.000000	0.000000	0.000000	0.000000	3.134661	NA	NA	NA	NA
preserved ~ pos	2062.764	0.897469	0.163843	0.052864	0.000682	-0.058976	NA	-	NA	NA
								0.0298709		
preserved ~ stimlen	2063.397	1.530789	0.446515	0.031665	0.000292	-0.058976	-	NA	NA	NA
							0.0252693			
preserved ~ stimlen + pos	2064.677	2.810504	0.024530	0.010878	0.000732	-0.058976	-	-	NA	NA
							0.0118939	0.0262023		
preserved ~ I(pos^2) + pos	2064.732	2.871927	0.023788	0.008519	0.000700	-0.058976	NA	-	NA	0.0020174
								0.0474951		
preserved ~ stimlen * pos	2066.481	4.614930	0.009951	0.002035	0.000860	-0.058976	-	-	0.0086946	NA
							0.0400111	0.1000424		
preserved ~ stimlen + I(pos^2)	2066.631	4.764664	0.009233	0.003306	0.000762	-0.058976	-	-	NA	0.0027432
							0.0134082	0.0497058		
+ pos										
preserved ~ stimlen * (I(pos^2) + pos)	2070.449	8.583118	0.001368	0.000490	0.000880	-0.058976	-	-	0.0209730	0.0124451
							0.0589760	-0.1975380		0.0015194

```
# plot no fragment data
```

```
NoFragData$LPFitted<-fitted(NoFragBestLPModel)
nofrag_obs_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData, NoFragData$patient[1],
  NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
```

```
nofrag_fitted_pos_len_summary <- NoFragData %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPFit
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
nofrag_fitted_pos_len_table <- nofrag_fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_f
nofrag_fitted_pos_len_table
```

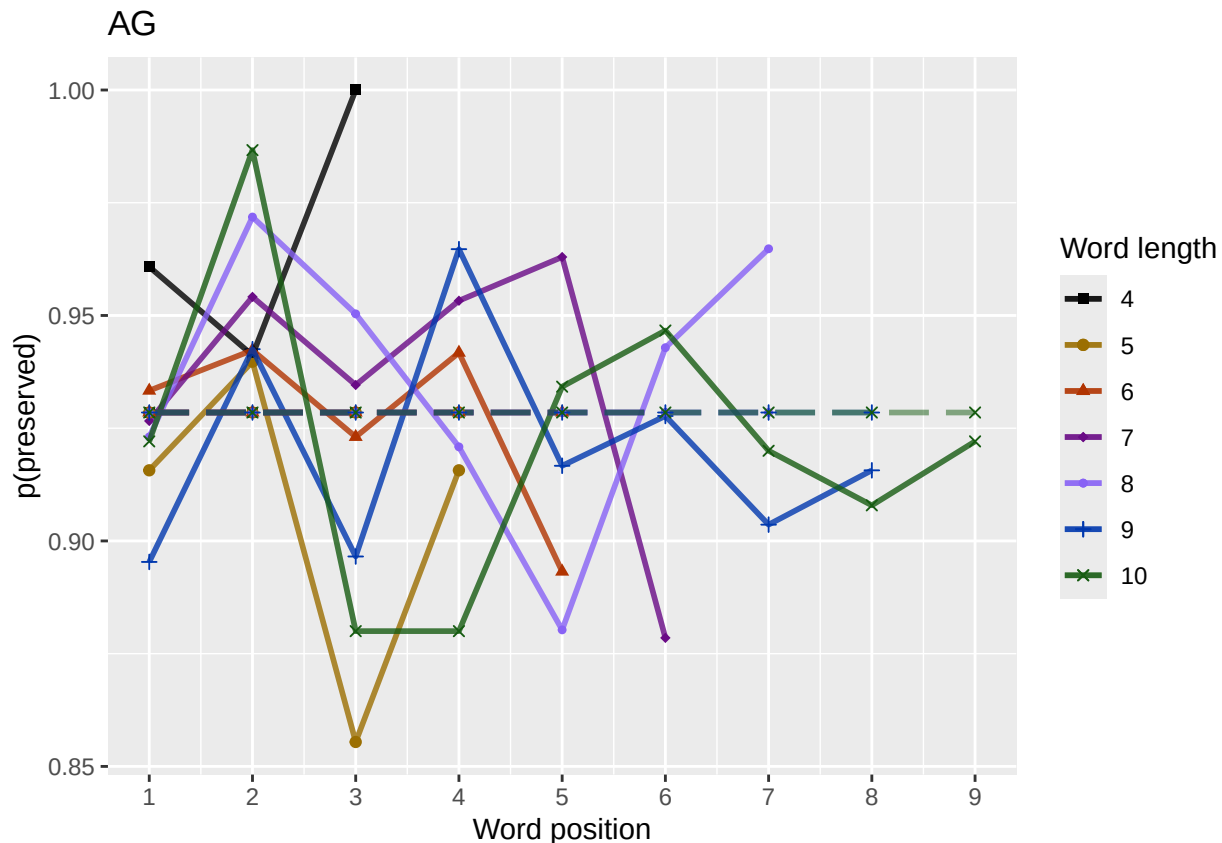
```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1      4 0.928 0.928 0.928 NA      NA      NA      NA      NA      NA
## 2      5 0.928 0.928 0.928 0.928 NA      NA      NA      NA      NA
## 3      6 0.928 0.928 0.928 0.928 0.928 NA      NA      NA      NA
## 4      7 0.928 0.928 0.928 0.928 0.928 0.928 NA      NA      NA
## 5      8 0.928 0.928 0.928 0.928 0.928 0.928 0.928 NA      NA
## 6      9 0.928 0.928 0.928 0.928 0.928 0.928 0.928 0.928 NA
## 7     10 0.928 0.928 0.928 0.928 0.928 0.928 0.928 0.928 0.928
```

```
fitted_pos_len_summary$stimlen<-factor(nofrag_fitted_pos_len_summary$stimlen)
fitted_pos_len_summary$pos<-factor(nofrag_fitted_pos_len_summary$pos)
# fitted_len_pos_plot <- ggplot(pos_len_summary,aes(x=pos,y=preserved,group=stimlen,shape=stimlen,color=stimlen))
# fitted_len_pos_plot <- fitted_len_pos_plot + geom_line(data=fitted_pos_len_summary, aes(x=pos,y=fitted,group=stimlen,shape=stimlen))
# geom_point(data=fitted_pos_len_summary,aes(x=pos,y=fitted,group=stimlen,shape=stimlen)) + ggtitle(paste0("Patient",patient[1]))
```

```
nofrag_fitted_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData,
  paste0(NoFragData$patient[1]),
  "LPFitted",
  NULL,
  palette_values,
  shape_values,
  obs_linetypes,
  pred_linetypes = c("longdash")
)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,
  "no_fragments_percent_preserved_by_length_pos_wfit.png"),
  plot=nofrag_fitted_len_pos_plot,
  device="png",unit="cm",
  width=15,height=11)
nofrag_fitted_len_pos_plot
```

back to full data

```
results_report_DF <- AddReportLine(results_report_DF,"min preserved",min_preserved)
results_report_DF <- AddReportLine(results_report_DF,"max preserved",max_preserved)
print(sprintf("Min/max preserved range: %.2f - %.2f",min_preserved,max_preserved))
```

```
## [1] "Min/max preserved range: 0.84 - 1.01"
```

```
# find the average difference between lengths and the average difference
# between positions taking the other factor into account
```

```
# choose fitted or raw -- we use fitted values because they are smoothed averages
# that take into account the influence of the other variable
```

```
table_to_use <- fitted_pos_len_table
# take away column with length information
table_to_use <- table_to_use[,2:ncol(table_to_use)]
```

```
# take averages for positions
# don't want downward estimates influenced by return upward of U
# therefore, for downward influence, use only the values before the min
# take the difference between each value (differences between position proportion correct) **NOTE** pro
# average the difference in probabilities
```

```
# in case min is close to the end or we are not using a min (for non-U-shaped)
# functions, we need to get differences _first_ and then average those (e.g.
# if there is an effect of length and we just average position data,
# later positions) will have a lower average (because of the length effect)
# even if all position functions go upward
```

```

table_pos_diffs <- t(diff(t(as.matrix(table_to_use))))
ncol_pos_diff_matrix <- ncol(table_pos_diffs)
first_col_mean <- mean(as.numeric(unlist(table_pos_diffs[,1])))
potential_u_shape <- 0
if(first_col_mean < 0){ # downward, so potential U-shape
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs >= 0] <- NA
  potential_u_shape <- 1
}else{
  # upward - use the positive values
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs <= 0] <- NA
}
average_pos_diffs <- apply(filtered_table_pos_diffs,2,mean,na.rm=TRUE)
OA_mean_pos_diff <- mean(average_pos_diffs,na.rm=TRUE)

table_len_diffs <- diff(as.matrix(table_to_use))
average_len_diffs <- apply(table_len_diffs,1,mean,na.rm=TRUE)
OA_mean_len_diff <- mean(average_len_diffs,na.rm=TRUE)
CurrentLabel<-"mean change in probability for each additional length: "
print(CurrentLabel)

## [1] "mean change in probability for each additional length: "
print(OA_mean_len_diff)

## [1] 0
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_len_diff)

CurrentLabel<-"mean change in probability for each additional position (excluding U-shape): "
print(CurrentLabel)

## [1] "mean change in probability for each additional position (excluding U-shape): "
print(OA_mean_pos_diff)

## [1] -0.004704731
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_pos_diff)

if(potential_u_shape){ # downward, so potential U-shape
  # take only positive values from the table
  filtered_pos_upward_u<-table_pos_diffs
  filtered_pos_upward_u[table_pos_diffs <= 0] <- NA
  average_pos_u_diffs <- apply(filtered_pos_upward_u,
                              2,mean,na.rm=TRUE)
  OA_mean_pos_u_diff <- mean(average_pos_u_diffs,na.rm=TRUE)
  if(is.nan(OA_mean_pos_u_diff) | (OA_mean_pos_u_diff <= 0)){
    print("No U-shape in this participant")
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
    potential_u_shape <- FALSE
  }else{
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 1)
  }
}

```

```

    CurrentLabel<-"Average upward change after U minimum"
    print(CurrentLabel)
    print(OA_mean_pos_u_diff)
    results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, OA_mean_pos_u_diff)

    CurrentLabel<-"Proportion of average downward change"
    prop_ave_down <- abs(OA_mean_pos_u_diff/OA_mean_pos_diff)
    results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, prop_ave_down)
  }
}else{
  print("No U-shape in this participant")
  results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
}

## [1] "No U-shape in this participant"
if(potential_u_shape){
  n_rows<-nrow(table_to_use)
  downward_dist <- numeric(n_rows)
  upward_dist <- numeric(n_rows)
  for(i in seq(1,n_rows)){
    current_row <- as.numeric(unlist(table_to_use[i,1:9]))
    current_row <- current_row[!is.na(current_row)]
    current_row_len <- length(current_row)
    row_min <- min(current_row,na.rm=TRUE)
    min_pos <- which(current_row == row_min)
    left_max <- max(current_row[1:min_pos],na.rm=TRUE)
    right_max <- max(current_row[min_pos:current_row_len])
    left_diff <- left_max - row_min
    right_diff <- right_max - row_min
    downward_dist[i]<-left_diff
    upward_dist[i]<-right_diff
  }
  print("differences from left max to min for each row: ")
  print(downward_dist)
  print("differences from min to right max for each row: ")
  print(upward_dist)

  # Use the max return upward (min of upward_dist) and then the
  # downward distance from the left for the same row

  print(" ")
  biggest_return_upward_row <- which(upward_dist == max(upward_dist))
  CurrentLabel <- "row with biggest return upward"
  print(CurrentLabel)
  print(biggest_return_upward_row)
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, biggest_return_upward_row)

  print(" ")
  CurrentLabel<-"downward distance for row with the largest upward value"
  print(CurrentLabel)
  print(downward_dist[biggest_return_upward_row])
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, downward_dist[biggest_return_upward_row])
}

```

```

CurrentLabel<-"return upward value"
print(upward_dist[biggest_return_upward_row])
results_report_DF <- AddReportLine(results_report_DF,
                                   CurrentLabel,
                                   upward_dist[biggest_return_upward_row])

print(" ")
# percentage return upward
percentage_return_upward <- upward_dist[biggest_return_upward_row]/downward_dist[biggest_return_upward_row]
CurrentLabel <- "Return upward as a proportion of the downward distance:"
print(CurrentLabel)
print(percentage_return_upward)
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,
                                   percentage_return_upward)
}else{
  print("no U-shape in this participant")
}

## [1] "no U-shape in this participant"

FLPModelEquations<-c(
  # models with log frequency, but no three-way interactions (due to difficulty interpreting and small sample sizes)
  "preserved ~ stimlen*log_freq",
  "preserved ~ stimlen+log_freq",
  "preserved ~ pos*log_freq",
  "preserved ~ pos+log_freq",
  "preserved ~ stimlen*log_freq + pos*log_freq",
  "preserved ~ stimlen*log_freq + pos",
  "preserved ~ stimlen + pos*log_freq",
  "preserved ~ stimlen + pos + log_freq",
  "preserved ~ (I(pos^2)+pos)*log_freq",
  "preserved ~ stimlen*log_freq + (I(pos^2) + pos)*log_freq",
  "preserved ~ stimlen*log_freq + I(pos^2) + pos",
  "preserved ~ stimlen + (I(pos^2) + pos)*log_freq",
  "preserved ~ stimlen + I(pos^2) + pos + log_freq",

  # models without frequency
  "preserved ~ 1",
  "preserved ~ stimlen",
  "preserved ~ pos",
  "preserved ~ stimlen + pos",
  "preserved ~ stimlen*pos",
  "preserved ~ I(pos^2)+pos",
  "preserved ~ stimlen + I(pos^2) + pos",
  "preserved ~ stimlen * (I(pos^2) + pos)"
)

FLPRes<-TestModels(FLPModelEquations,PosDat)

## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##

```

```

## Coefficients:
## (Intercept)          pos      log_freq
##      2.70700      -0.04976      0.12405
##
## Degrees of Freedom: 4021 Total (i.e. Null);  4019 Residual
## Null Deviance:      2180
## Residual Deviance: 2159  AIC: 2165
## log likelihood:  -1079.339
## Nagelkerke R2:  0.01236317
## % pres/err predicted correctly:  -566.5224
## % of predictable range [ (model-null)/(1-null) ]:  0.005255195
## *****
## model index:  8
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos      log_freq
##      2.37329      0.05095     -0.06427      0.13527
##
## Degrees of Freedom: 4021 Total (i.e. Null);  4018 Residual
## Null Deviance:      2180
## Residual Deviance: 2157  AIC: 2165
## log likelihood:  -1078.581
## Nagelkerke R2:  0.01325972
## % pres/err predicted correctly:  -566.2444
## % of predictable range [ (model-null)/(1-null) ]:  0.005742482
## *****
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      log_freq          pos  stimlen:log_freq
##      2.36636      0.04925      0.31878     -0.06417     -0.02352
##
## Degrees of Freedom: 4021 Total (i.e. Null);  4017 Residual
## Null Deviance:      2180
## Residual Deviance: 2156  AIC: 2166
## log likelihood:  -1077.856
## Nagelkerke R2:  0.01411656
## % pres/err predicted correctly:  -566.042
## % of predictable range [ (model-null)/(1-null) ]:  0.006097275
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)          pos      log_freq  pos:log_freq
##      2.696990     -0.046518      0.099777      0.006018

```

```

##
## Degrees of Freedom: 4021 Total (i.e. Null); 4018 Residual
## Null Deviance: 2180
## Residual Deviance: 2159 AIC: 2167
## log likelihood: -1079.251
## Nagelkerke R2: 0.01246732
## % pres/err predicted correctly: -566.4903
## % of predictable range [ (model-null)/(1-null) ]: 0.00531156
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq pos stimlen:log_freq log_fr
## 2.36723 0.04493 0.32251 -0.05599 -0.03134 0
##
## Degrees of Freedom: 4021 Total (i.e. Null); 4016 Residual
## Null Deviance: 2180
## Residual Deviance: 2155 AIC: 2167
## log likelihood: -1077.481
## Nagelkerke R2: 0.01455913
## % pres/err predicted correctly: -565.9391
## % of predictable range [ (model-null)/(1-null) ]: 0.006277623
## *****
## model index: 7
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen pos log_freq pos:log_freq
## 2.375231 0.049595 -0.061662 0.118599 0.004058
##
## Degrees of Freedom: 4021 Total (i.e. Null); 4017 Residual
## Null Deviance: 2180
## Residual Deviance: 2157 AIC: 2167
## log likelihood: -1078.542
## Nagelkerke R2: 0.01330583
## % pres/err predicted correctly: -566.2348
## % of predictable range [ (model-null)/(1-null) ]: 0.005759435
## *****
## model index: 13
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen I(pos^2) pos log_freq
## 2.406274 0.049906 0.001776 -0.079693 0.135224
##
## Degrees of Freedom: 4021 Total (i.e. Null); 4017 Residual
## Null Deviance: 2180

```

```

## Residual Deviance: 2157 AIC: 2167
## log likelihood: -1078.57
## Nagelkerke R2: 0.01327206
## % pres/err predicted correctly: -566.2372
## % of predictable range [ (model-null)/(1-null) ]: 0.005755239
## *****
## model index: 11
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq I(pos^2) pos stimlen:log_freq
## 2.3847340 0.0486711 0.3181842 0.0009861 -0.0727331 -0.0727331
##
## Degrees of Freedom: 4021 Total (i.e. Null); 4016 Residual
## Null Deviance: 2180
## Residual Deviance: 2156 AIC: 2168
## log likelihood: -1077.853
## Nagelkerke R2: 0.01412034
## % pres/err predicted correctly: -566.0389
## % of predictable range [ (model-null)/(1-null) ]: 0.006102763
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq
## 2.3811 0.0169 0.1350
##
## Degrees of Freedom: 4021 Total (i.e. Null); 4019 Residual
## Null Deviance: 2180
## Residual Deviance: 2162 AIC: 2168
## log likelihood: -1080.899
## Nagelkerke R2: 0.01051866
## % pres/err predicted correctly: -566.9975
## % of predictable range [ (model-null)/(1-null) ]: 0.004422472
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq stimlen:log_freq
## 2.37424 0.01524 0.31932 -0.02362
##
## Degrees of Freedom: 4021 Total (i.e. Null); 4018 Residual
## Null Deviance: 2180
## Residual Deviance: 2160 AIC: 2168
## log likelihood: -1080.167
## Nagelkerke R2: 0.01138479

```

```

## % pres/err predicted correctly: -566.7846
## % of predictable range [ (model-null)/(1-null) ]: 0.004795648
## *****
## model index: 9
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos log_freq I(pos^2):log_freq
## 2.731182 0.001289 -0.061725 -0.016453 -0.007789
##
## Degrees of Freedom: 4021 Total (i.e. Null); 4016 Residual
## Null Deviance: 2180
## Residual Deviance: 2157 AIC: 2169
## log likelihood: -1078.432
## Nagelkerke R2: 0.01343629
## % pres/err predicted correctly: -566.2494
## % of predictable range [ (model-null)/(1-null) ]: 0.005733694
## *****
## model index: 12
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen I(pos^2) pos log_freq I(pos
## 2.366e+00 5.003e-02 -1.549e-03 -5.216e-02 -5.118e-05
##
## Degrees of Freedom: 4021 Total (i.e. Null); 4015 Residual
## Null Deviance: 2180
## Residual Deviance: 2155 AIC: 2169
## log likelihood: -1077.736
## Nagelkerke R2: 0.0142577
## % pres/err predicted correctly: -566.0135
## % of predictable range [ (model-null)/(1-null) ]: 0.00614717
## *****
## model index: 10
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq I(pos^2) pos stim
## 2.368114 0.045429 0.197632 -0.000900 -0.051913
## log_freq:pos
## 0.073277
##
## Degrees of Freedom: 4021 Total (i.e. Null); 4014 Residual
## Null Deviance: 2180
## Residual Deviance: 2154 AIC: 2170
## log likelihood: -1076.93
## Nagelkerke R2: 0.01521019
## % pres/err predicted correctly: -565.7874

```



```

## % of predictable range [ (model-null)/(1-null) ]: 0.006543641
## *****
## model index: 16
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##    2.73419      -0.06247
##
## Degrees of Freedom: 4021 Total (i.e. Null); 4020 Residual
## Null Deviance: 2180
## Residual Deviance: 2174 AIC: 2178
## log likelihood: -1087.135
## Nagelkerke R2: 0.003126797
## % pres/err predicted correctly: -568.7646
## % of predictable range [ (model-null)/(1-null) ]: 0.001325101
## *****
## model index: 19
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##    2.770121    0.002423    -0.083995
##
## Degrees of Freedom: 4021 Total (i.e. Null); 4019 Residual
## Null Deviance: 2180
## Residual Deviance: 2174 AIC: 2180
## log likelihood: -1087.115
## Nagelkerke R2: 0.003150882
## % pres/err predicted correctly: -568.7586
## % of predictable range [ (model-null)/(1-null) ]: 0.001335752
## *****
## model index: 17
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##    2.708331    0.004004    -0.063750
##
## Degrees of Freedom: 4021 Total (i.e. Null); 4019 Residual
## Null Deviance: 2180
## Residual Deviance: 2174 AIC: 2180
## log likelihood: -1087.13
## Nagelkerke R2: 0.003132907
## % pres/err predicted correctly: -568.7622
## % of predictable range [ (model-null)/(1-null) ]: 0.001329402
## *****
## model index: 18

```

```

##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      pos  stimlen:pos
##      3.21330      -0.05742      -0.21998      0.01840
##
## Degrees of Freedom: 4021 Total (i.e. Null);  4018 Residual
## Null Deviance:      2180
## Residual Deviance: 2173 AIC: 2181
## log likelihood: -1086.657
## Nagelkerke R2:  0.003694015
## % pres/err predicted correctly: -568.6275
## % of predictable range [ (model-null)/(1-null) ]:  0.001565498
## *****
## model index:  14
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
##      2.486
##
## Degrees of Freedom: 4021 Total (i.e. Null);  4021 Residual
## Null Deviance:      2180
## Residual Deviance: 2180 AIC: 2182
## log likelihood: -1089.768
## Nagelkerke R2:  2.653751e-16
## % pres/err predicted correctly: -569.5206
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****
## model index:  20
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)      pos
##      2.750672      0.002680      0.002279      -0.083564
##
## Degrees of Freedom: 4021 Total (i.e. Null);  4018 Residual
## Null Deviance:      2180
## Residual Deviance: 2174 AIC: 2182
## log likelihood: -1087.113
## Nagelkerke R2:  0.003153534
## % pres/err predicted correctly: -568.7572
## % of predictable range [ (model-null)/(1-null) ]:  0.001338067
## *****
## model index:  15
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)

```

```
##
## Coefficients:
## (Intercept)      stimlen
##      2.71570      -0.02971
##
## Degrees of Freedom: 4021 Total (i.e. Null);  4020 Residual
## Null Deviance:      2180
## Residual Deviance: 2179  AIC: 2183
## log likelihood:  -1089.421
## Nagelkerke R2:  0.0004123246
## % pres/err predicted correctly:  -569.4232
## % of predictable range [ (model-null)/(1-null) ]:  0.0001708448
## *****
## model index:  21
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)      pos  stimlen:I(pos^2)      stimlen:I(pos^2)
##      2.864530      -0.030138      -0.046046      0.066079      0.004261      -0.000000
##
## Degrees of Freedom: 4021 Total (i.e. Null);  4016 Residual
## Null Deviance:      2180
## Residual Deviance: 2173  AIC: 2185
## log likelihood:  -1086.404
## Nagelkerke R2:  0.00399498
## % pres/err predicted correctly:  -568.554
## % of predictable range [ (model-null)/(1-null) ]:  0.00169423
## *****
```

```
BestFLPModel<-FLPRes$ModelResult[[1]]
BestFLPModelFormula<-FLPRes$Model[[1]]

FLPAICSummary<-data.frame(Model=FLPRes$Model,
                          AIC=FLPRes$AIC,row.names=FLPRes$Model)
FLPAICSummary$DeltaAIC<-FLPAICSummary$AIC-FLPAICSummary$AIC[1]
FLPAICSummary$AICexp<-exp(-0.5*FLPAICSummary$DeltaAIC)
FLPAICSummary$AICwt<-FLPAICSummary$AICexp/sum(FLPAICSummary$AICexp)
FLPAICSummary$NagR2<-FLPRes$NagR2

FLPAICSummary <- merge(FLPAICSummary,FLPRes$CoefficientValues,
                      by='row.names',sort=FALSE)
FLPAICSummary <- subset(FLPAICSummary, select = -c(Row.names))

write.csv(FLPAICSummary,paste0(TablesDir,CurPat,"_",
                              RunLabel,"_",CurTask,
                              "_freq_len_pos_model_summary.csv"),row.names = FALSE)

kable(FLPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	log_stimlen	log_pos	log_freq	I(pos^2)	pos	stimlen:I(pos^2)	stimlen:I(pos^2)
preserved ~	2164.67	0.000000	1.000000	0.18412	0.23770	0.12405	NA	NA	NA	NA	NA	NA	NA
pos +											0.0497582		
log_freq													

Model	AIC Delta	AIC	AICw	NagR	R ²	Intercept	log_freq	stimlen	log_pos	log_freq	I(pos)	pos^2	log_freq	I(pos)	stimlen:I(pos^2)
preserved ~ stimlen + pos + log_freq	2165.1623	307856	2976	2159	23973	2925	00458	2696	-	NA	NA	NA	NA	NA	NA
									0.0642699						
preserved ~ stimlen * log_freq + pos	2165.7023	42966	3270	2003	21366	3574	9051	87835	-	NA	NA	NA	NA	NA	NA
									0.0235008	0.0641740					
preserved ~ pos * log_freq	2166.5623	38386	7328	7478	2169	990	0.0997	7747	-	0.0060	NA	NA	NA	NA	NA
									0.0465183						
preserved ~ stimlen * log_freq + pos *	2166.2623	40438	9070	2701	2123	3672	2744	9302	-	NA	0.0139	NA	NA	NA	NA
									0.0313406	0.0559940					
preserved ~ stimlen + pos * log_freq	2167.2140	50739	4005	6013	2305	8230	4959	4859	-	0.0040	NA	NA	NA	NA	NA
									0.0616619						
preserved ~ stimlen + I(pos^2) + pos + log_freq	2167.2462	41261	0393	7683	2706	2704	9905	5522	-	NA	NA	0.0017	NA	NA	NA
									0.0796933						
preserved ~ stimlen * log_freq + I(pos^2) + pos	2167.3026	8327	0365	8084	9123	8173	4486	7118	-	NA	NA	0.0009	NA	NA	NA
									0.0234402	0.0727331					
preserved ~ stimlen + log_freq	2167.3978	5325	0284	5928	8318	7106	1689	9349	NA	NA	NA	NA	NA	NA	NA
preserved ~ stimlen * log_freq	2168.3334	4886	8535	0391	2384	2361	5035	1932	NA	NA	NA	NA	NA	NA	NA
									0.0236235						
preserved ~ (I(pos^2) + pos) * log_freq	2168.4634	6212	3407	2695	3823	7318	-	NA	-	0.0754	NA	0.0012	NA	NA	NA
									0.0164534	0.0617252		0.0077890			
preserved ~ stimlen + (I(pos^2) + pos) * log_freq	2169.4794	6239	0075	9870	3256	5005	00279	NA	-	0.0754	NA	-	NA	NA	NA
									0.0000512	0.0521563		0.0016492	0.0080605		
preserved ~ stimlen * log_freq + (I(pos^2) + pos) * log_freq	2169.5668	1683	7405	6637	1152	3680	1445	0287	-	NA	0.0732	NA	-	NA	NA
									0.0276775	0.0519128		0.0009000	0.0068016		

Model	AIC Delta	AIC	AICw	NagR ²	(Intercept)	log_stimlen	log_freq	log_pos	log_freq(I(pos^2))	log_freq(I(pos^2) + pos)	log_freq(I(pos^2) + pos)	log_freq(I(pos^2) + pos)	log_freq(I(pos^2) + pos)	log_freq(I(pos^2) + pos)	log_freq(I(pos^2) + pos)
preserved ~ pos	2178.27	5900020	1.82	0.0032	2.70700	NA	NA	-	NA	NA	NA	NA	NA	NA	NA
								0.0624749							
preserved ~ I(pos^2) + pos	2180.23	5504810	1.08	0.0032	2.70700	NA	NA	-	NA	NA	0.0024	NA	NA	NA	NA
								0.0839953							
preserved ~ stimlen + pos	2180.26	5807070	1.35	0.0032	2.70700	NA	NA	-	NA	NA	NA	NA	NA	NA	NA
								0.0637498							
preserved ~ stimlen * pos	2181.31	5362730	2.40	0.0033	2.70700	NA	NA	-	NA	NA	NA	NA	NA	0.0184	NA
								0.0574240						0.2199820	
preserved ~ 1	2181.53	5567900	1.86	0.0020	2.70700	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
preserved ~ stimlen + I(pos^2) + pos	2182.22	5470100	1.50	0.0033	2.70700	NA	NA	-	NA	NA	0.0022	NA	NA	NA	NA
								0.0835644							
preserved ~ stimlen	2182.81	1629350	1.88	0.0021	2.70700	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
								0.0297103							
preserved ~ stimlen * (I(pos^2) + pos)	2184.20	8290010	4.26	0.0039	2.70700	NA	NA	0.0660	NA	-	NA	NA	-	0.0042605	
								0.0301381				0.0460457		0.0061342	

```
print(BestFLPModelFormula)
```

```
## [1] "preserved ~ pos + log_freq"
```

```
print(BestFLPModel)
```

```
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      pos      log_freq
##      2.70700     -0.04976      0.12405
##
## Degrees of Freedom: 4021 Total (i.e. Null);  4019 Residual
## Null Deviance:      2180
## Residual Deviance: 2159  AIC: 2165
```

```
# do a median split on frequency to plot hf/ls effects (analysis is continuous)
median_freq <- median(PosDat$log_freq)
PosDat$freq_bin[PosDat$log_freq >= median_freq] <- "hf"
PosDat$freq_bin[PosDat$log_freq < median_freq] <- "lf"
```

```
PosDat$FLPFitted <- fitted(BestFLPModel)
```

```
HFDat <- PosDat[PosDat$freq_bin == "hf",]
LFDat <- PosDat[PosDat$freq_bin == "lf",]
```

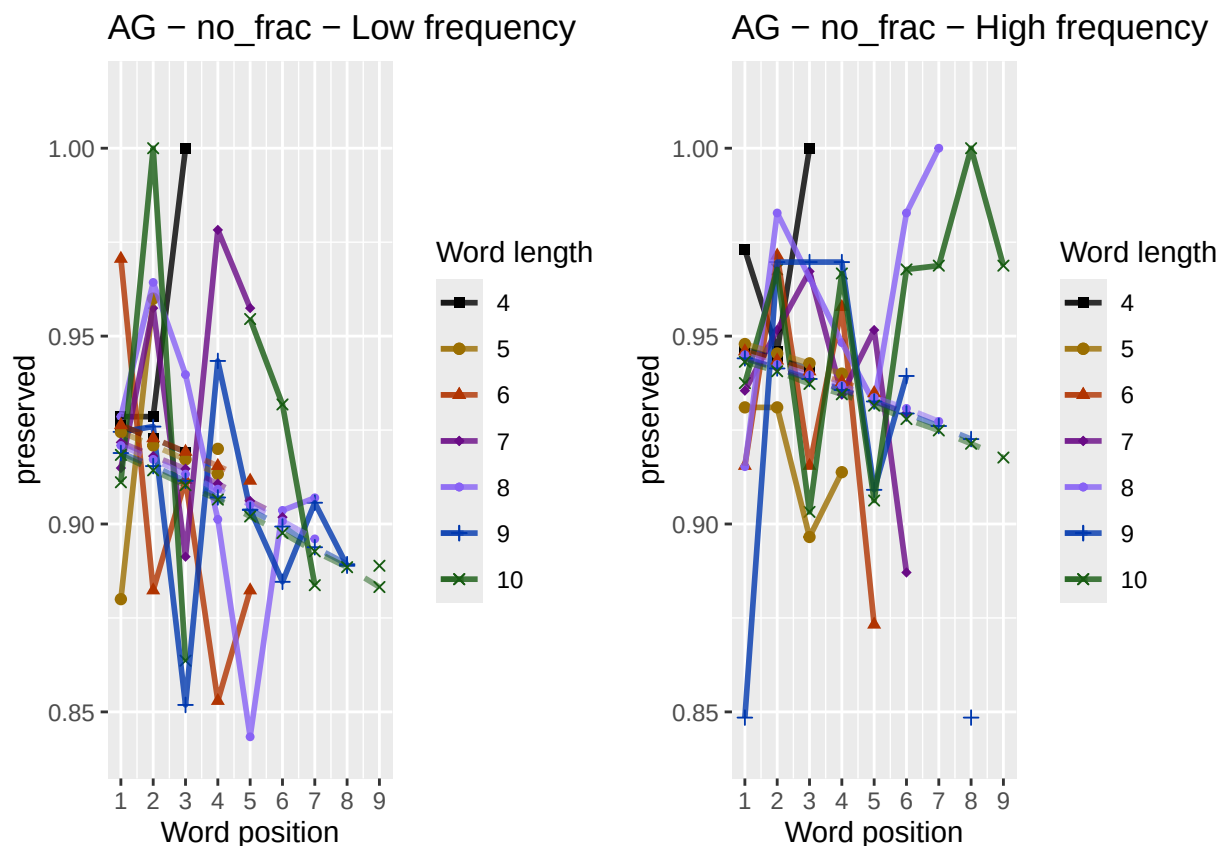
```
HF_Plot <- plot_len_pos_obs_predicted(HFDat, paste0(CurPat, " - ", RunLabel, " - High frequency"), "FLPFitted")
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
LF_Plot <- plot_len_pos_obs_predicted(LFdat, paste0(CurPat, " - ", RunLabel, " - Low frequency"), "FLPFitted")

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
library(ggpubr)
Both_Plots <- ggarrange(LF_Plot, HF_Plot) # labels=c("LF", "HF", ncol=2)

## Warning: Removed 4 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: Removed 1 row containing missing values or values outside the scale range (`geom_line()`).
## Warning: Removed 1 row containing missing values or values outside the scale range (`geom_point()`).
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_",
              CurTask, "_frequency_effect_length_pos_wfit.png"),
       device="png", unit="cm", width=30, height=11)
print(Both_Plots)
```



```
# only main effects
MEModelEquations<-c(
  "preserved ~ CumPres",
  "preserved ~ CumErr",
  "preserved ~ (I(pos^2)+pos)",
  "preserved ~ pos",
```

```

"preserved ~ stimlen",
"preserved ~ 1"
)
MERes<-TestModels(MEModelEquations,PosDat)

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.776      -0.879
##
## Degrees of Freedom: 4021 Total (i.e. Null); 4020 Residual
## Null Deviance:      2180
## Residual Deviance: 2052 AIC: 2056
## log likelihood: -1025.771
## Nagelkerke R2: 0.07486923
## % pres/err predicted correctly: -542.6556
## % of predictable range [ (model-null)/(1-null) ]: 0.04708872
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      2.30348      0.07334
##
## Degrees of Freedom: 4021 Total (i.e. Null); 4020 Residual
## Null Deviance:      2180
## Residual Deviance: 2173 AIC: 2177
## log likelihood: -1086.709
## Nagelkerke R2: 0.003632283
## % pres/err predicted correctly: -568.6887
## % of predictable range [ (model-null)/(1-null) ]: 0.00145824
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##      2.73419      -0.06247
##
## Degrees of Freedom: 4021 Total (i.e. Null); 4020 Residual
## Null Deviance:      2180
## Residual Deviance: 2174 AIC: 2178
## log likelihood: -1087.135
## Nagelkerke R2: 0.003126797

```

```

## % pres/err predicted correctly: -568.7646
## % of predictable range [ (model-null)/(1-null) ]: 0.001325101
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos
## 2.770121 0.002423 -0.083995
##
## Degrees of Freedom: 4021 Total (i.e. Null); 4019 Residual
## Null Deviance: 2180
## Residual Deviance: 2174 AIC: 2180
## log likelihood: -1087.115
## Nagelkerke R2: 0.003150882
## % pres/err predicted correctly: -568.7586
## % of predictable range [ (model-null)/(1-null) ]: 0.001335752
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 2.486
##
## Degrees of Freedom: 4021 Total (i.e. Null); 4021 Residual
## Null Deviance: 2180
## Residual Deviance: 2180 AIC: 2182
## log likelihood: -1089.768
## Nagelkerke R2: 2.653751e-16
## % pres/err predicted correctly: -569.5206
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen
## 2.71570 -0.02971
##
## Degrees of Freedom: 4021 Total (i.e. Null); 4020 Residual
## Null Deviance: 2180
## Residual Deviance: 2179 AIC: 2183
## log likelihood: -1089.421
## Nagelkerke R2: 0.0004123246
## % pres/err predicted correctly: -569.4232
## % of predictable range [ (model-null)/(1-null) ]: 0.0001708448
## *****

```



```

BestMEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
BestMEModelFormula<-MERes$Model[[BestModelIndexL1]]

MEAICSummary<-data.frame(Model=MERes$Model,
                          AIC=MERes$AIC,row.names=MERes$Model)
MEAICSummary$DeltaAIC<-MEAICSummary$AIC-MEAICSummary$AIC[1]
MEAICSummary$AICexp<-exp(-0.5*MEAICSummary$DeltaAIC)
MEAICSummary$AICwt<-MEAICSummary$AICexp/sum(MEAICSummary$AICexp)
MEAICSummary$NagR2<-MERes$NagR2

MEAICSummary <- merge(MEAICSummary,MERes$CoefficientValues,
                      by='row.names',sort=FALSE)
MEAICSummary <- subset(MEAICSummary, select = -c(Row.names))

write.csv(MEAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                             CurTask,"_main_effects_model_summary.csv"),
          row.names = FALSE)
kable(MEAICSummary)

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	2055.5420	0.0000	1	1	0.0748692	7.775734	NA	-	NA	NA	NA
preserved ~ CumPres	2177.4192	121.8769	0	0	0.0036323	23.303477	0.0733389	NA	NA	NA	NA
preserved ~ pos	2178.2711	122.7286	0	0	0.0031268	28.734188	NA	NA	NA	-	NA
										0.0624749	
preserved ~ (I(pos^2) + pos)	2180.2301	124.6881	0	0	0.0031509	29.770121	NA	NA	0.0024234	-	NA
										0.0839953	
preserved ~ 1	2181.5351	125.9934	0	0	0.0000000	0.486254	NA	NA	NA	NA	NA
preserved ~ stimlen	2182.8411	127.2995	0	0	0.0004123	7.715702	NA	NA	NA	NA	-
											0.0297103

```

if(DoSimulations){
  BestMEModelFormulaRnd <- BestMEModelFormula
  if(grepl("CumPres",BestMEModelFormulaRnd)){
    BestMEModelFormulaRnd <- gsub("CumPres","RndCumPres",BestMEModelFormulaRnd)
  }else if(grepl("CumErr",BestMEModelFormulaRnd)){
    BestMEModelFormulaRnd <- gsub("CumErr","RndCumErr",BestMEModelFormulaRnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestMEModelFormulaRnd),
                       family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
  }
  ModelNames<-c(paste0("***",BestMEModelFormula),
                rep(BestMEModelFormulaRnd,RandomSamples))
  AICValues <- c(BestMEModel$aic,RndModelAIC)
}

```

```

BestMEModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
BestMEModelRndDF <- BestMEModelRndDF %>% arrange(AIC)
BestMEModelRndDF <- rbind(BestMEModelRndDF,
                           data.frame(Name=c("Random average"),
                                         AIC=c(mean(RndModelAIC))))
BestMEModelRndDF <- rbind(BestMEModelRndDF,
                           data.frame(Name=c("Random SD"),
                                         AIC=c(sd(RndModelAIC))))

write.csv(BestMEModelRndDF,
          paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                 "_best_main_effects_model_with_random_cum_term.csv"),
          row.names = FALSE)
}

syll_component_summary <- PosDat %>%
  group_by(syll_component) %>% summarise(MeanPres = mean(preserved),
                                         N = n())
write.csv(syll_component_summary,paste0(TablesDir,CurPat,"_",
                                         RunLabel,"_",CurTask,
                                         "_syllable_component_summary.csv"),row.names = FALSE)
kable(syll_component_summary)

```

syll_component	MeanPres	N
1	0.9160305	524
O	0.9107527	1860
P	0.7333333	30
S	0.9458333	240
V	0.9429825	1368

```

# main effects models for data without satellite positions

keep_components = c("0","V","1")
OV1Data <- PosDat[PosDat$syll_component %in% keep_components,]
OV1Data <- OV1Data %>% select(stim_number,
                             stimlen,stim,pos,
                             preserved,syll_component)
OV1Data$CumPres <- CalcCumPres(OV1Data)
OV1Data$CumErr <- CalcCumErrFromPreserved(OV1Data)

SimpSyllMEAICSummary <- EvaluateSubsetData(OV1Data,MEModelEquations)

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.7735      -0.9184
##
## Degrees of Freedom: 3751 Total (i.e. Null); 3750 Residual

```

```

## Null Deviance:      2032
## Residual Deviance: 1907 AIC: 1911
## log likelihood:    -953.7335
## Nagelkerke R2:    0.07803731
## % pres/err predicted correctly:  -504.293
## % of predictable range [ (model-null)/(1-null) ]:  0.04982022
## *****
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      2.79009      -0.07538
##
## Degrees of Freedom: 3751 Total (i.e. Null);  3750 Residual
## Null Deviance:      2032
## Residual Deviance: 2025 AIC: 2029
## log likelihood:    -1012.372
## Nagelkerke R2:    0.004585372
## % pres/err predicted correctly:  -529.7616
## % of predictable range [ (model-null)/(1-null) ]:  0.001927736
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##      2.90402      0.00754      -0.14274
##
## Degrees of Freedom: 3751 Total (i.e. Null);  3749 Residual
## Null Deviance:      2032
## Residual Deviance: 2024 AIC: 2030
## log likelihood:    -1012.188
## Nagelkerke R2:    0.004819137
## % pres/err predicted correctly:  -529.7124
## % of predictable range [ (model-null)/(1-null) ]:  0.002020387
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      2.31149      0.07564
##
## Degrees of Freedom: 3751 Total (i.e. Null);  3750 Residual
## Null Deviance:      2032
## Residual Deviance: 2026 AIC: 2030
## log likelihood:    -1013.235

```

```
## Nagelkerke R2: 0.003487428
## % pres/err predicted correctly: -530.0388
## % of predictable range [ (model-null)/(1-null) ]: 0.001406569
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 2.487
##
## Degrees of Freedom: 3751 Total (i.e. Null); 3751 Residual
## Null Deviance: 2032
## Residual Deviance: 2032 AIC: 2034
## log likelihood: -1015.973
## Nagelkerke R2: 7.965021e-16
## % pres/err predicted correctly: -530.7868
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen
## 2.87063 -0.04948
##
## Degrees of Freedom: 3751 Total (i.e. Null); 3750 Residual
## Null Deviance: 2032
## Residual Deviance: 2030 AIC: 2034
## log likelihood: -1015.08
## Nagelkerke R2: 0.0011381
## % pres/err predicted correctly: -530.5371
## % of predictable range [ (model-null)/(1-null) ]: 0.000469467
## *****
```

```
write.csv(SimpSyllMEAICSummary,
  paste0(TablesDir, CurPat, "_", RunLabel, "_",
    CurTask,
    "_OV1_main_effects_model_summary.csv"),
  row.names = FALSE)
kable(SimpSyllMEAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	1911.467	0.0000	1	1	0.0780373	3.773520	NA	-	NA	NA	NA
preserved ~ pos	2028.744	17.2773	0	0	0.0045852	1.790092	NA	NA	NA	-	NA
										0.0753812	
preserved ~ (I(pos^2) + pos)	2030.377	18.9098	0	0	0.0048192	1.904018	NA	NA	0.0075398	-	NA
										0.1427445	

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumPres	2030.470	19.0028	0	0	0.003487	2.311485	0.0756362	NA	NA	NA	NA
preserved ~ 1	2033.945	22.4784	0	0	0.000000	0.487219	NA	NA	NA	NA	NA
preserved ~ stimlen	2034.159	22.6923	0	0	0.001138	2.870633	NA	NA	NA	NA	-
											0.0494846

```
# main effects models for data without satellite or coda positions
# (tradeoff -- takes away more complex segments, in addition to satellites, but
# also reduces data)
```

```
keep_components = c("0","V")
OVDData <- PosDat[PosDat$syll_component %in% keep_components,]
OVDData <- OVDData %>% select(stim_number,
                             stimlen,stim,pos,
                             preserved,syll_component)
OVDData$CumPres <- CalcCumPres(OVDData)
OVDData$CumErr <- CalcCumErrFromPreserved(OVDData)

SimpSyllMEAICSummary2<-EvaluateSubsetData(OVDData,MEModelEquations)
```

```
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.765      -1.028
##
## Degrees of Freedom: 3227 Total (i.e. Null); 3226 Residual
## Null Deviance:      1729
## Residual Deviance: 1632 AIC: 1636
## log likelihood: -815.7756
## Nagelkerke R2: 0.07192301
## % pres/err predicted correctly: -428.5594
## % of predictable range [ (model-null)/(1-null) ]: 0.04777823
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##      2.83057      -0.08128
##
## Degrees of Freedom: 3227 Total (i.e. Null); 3226 Residual
## Null Deviance:      1729
## Residual Deviance: 1722 AIC: 1726
## log likelihood: -860.8868
## Nagelkerke R2: 0.005621657
```

```

## % pres/err predicted correctly: -449.0447
## % of predictable range [ (model-null)/(1-null) ]: 0.002367613
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos
## 2.880053 0.003367 -0.111292
##
## Degrees of Freedom: 3227 Total (i.e. Null); 3225 Residual
## Null Deviance: 1729
## Residual Deviance: 1722 AIC: 1728
## log likelihood: -860.8528
## Nagelkerke R2: 0.005672329
## % pres/err predicted correctly: -449.0344
## % of predictable range [ (model-null)/(1-null) ]: 0.002390522
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 2.504
##
## Degrees of Freedom: 3227 Total (i.e. Null); 3227 Residual
## Null Deviance: 1729
## Residual Deviance: 1729 AIC: 1731
## log likelihood: -864.6544
## Nagelkerke R2: 2.676829e-16
## % pres/err predicted correctly: -450.1128
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumPres
## 2.39602 0.05419
##
## Degrees of Freedom: 3227 Total (i.e. Null); 3226 Residual
## Null Deviance: 1729
## Residual Deviance: 1727 AIC: 1731
## log likelihood: -863.7228
## Nagelkerke R2: 0.0013912
## % pres/err predicted correctly: -449.8564
## % of predictable range [ (model-null)/(1-null) ]: 0.0005682448
## *****

```

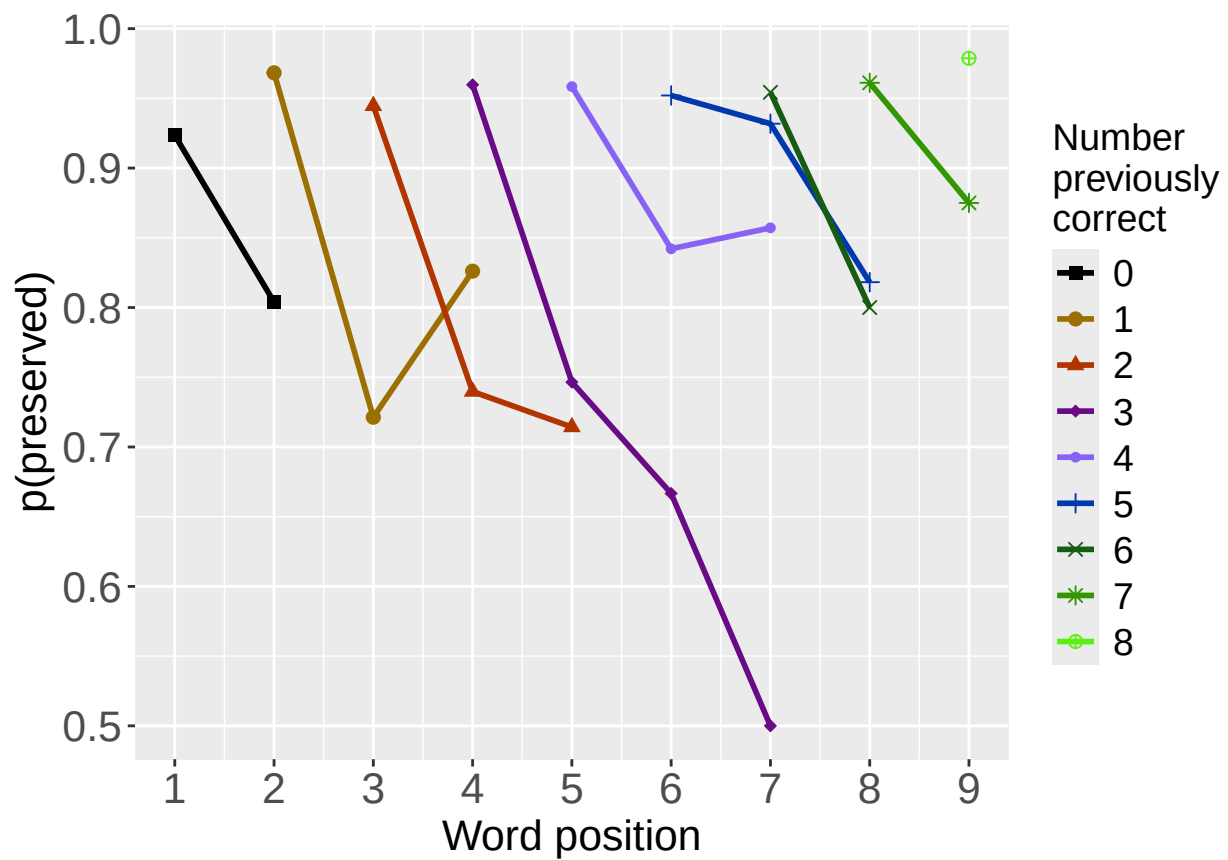
```
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      2.76745      -0.03417
##
## Degrees of Freedom: 3227 Total (i.e. Null); 3226 Residual
## Null Deviance:      1729
## Residual Deviance: 1729 AIC: 1733
## log likelihood: -864.2879
## Nagelkerke R2: 0.0005473472
## % pres/err predicted correctly: -450.0139
## % of predictable range [ (model-null)/(1-null) ]: 0.000219205
## *****
```

```
write.csv(SimpSyllMEAICSummary2,
          paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
                "_OV_main_effects_model_summary.csv"), row.names = FALSE)
kable(SimpSyllMEAICSummary2)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	1635.551	0.00000	1	1	0.071923	0.765296	NA	-	NA	NA	NA
preserved ~ pos	1725.774	0.22241	0	0	0.005621	2.830565	NA	NA	NA	-	NA
										0.0812832	
preserved ~ (I(pos^2) + pos)	1727.706	0.21544	0	0	0.005672	3.880053	NA	NA	0.0033672	-	NA
										0.1112923	
preserved ~ 1	1731.309	5.75760	0	0	0.000000	0.503852	NA	NA	NA	NA	NA
preserved ~ CumPres	1731.446	5.89450	0	0	0.001391	2.396016	0.0541875	NA	NA	NA	NA
preserved ~ stimlen	1732.576	7.02472	0	0	0.000547	3.767452	NA	NA	NA	NA	-
											0.0341682

```
# plot prev err and prev cor plots
PrevCorPlot<-PlotObsPreviousCorrect(PosDat,palette_values,shape_values)
```

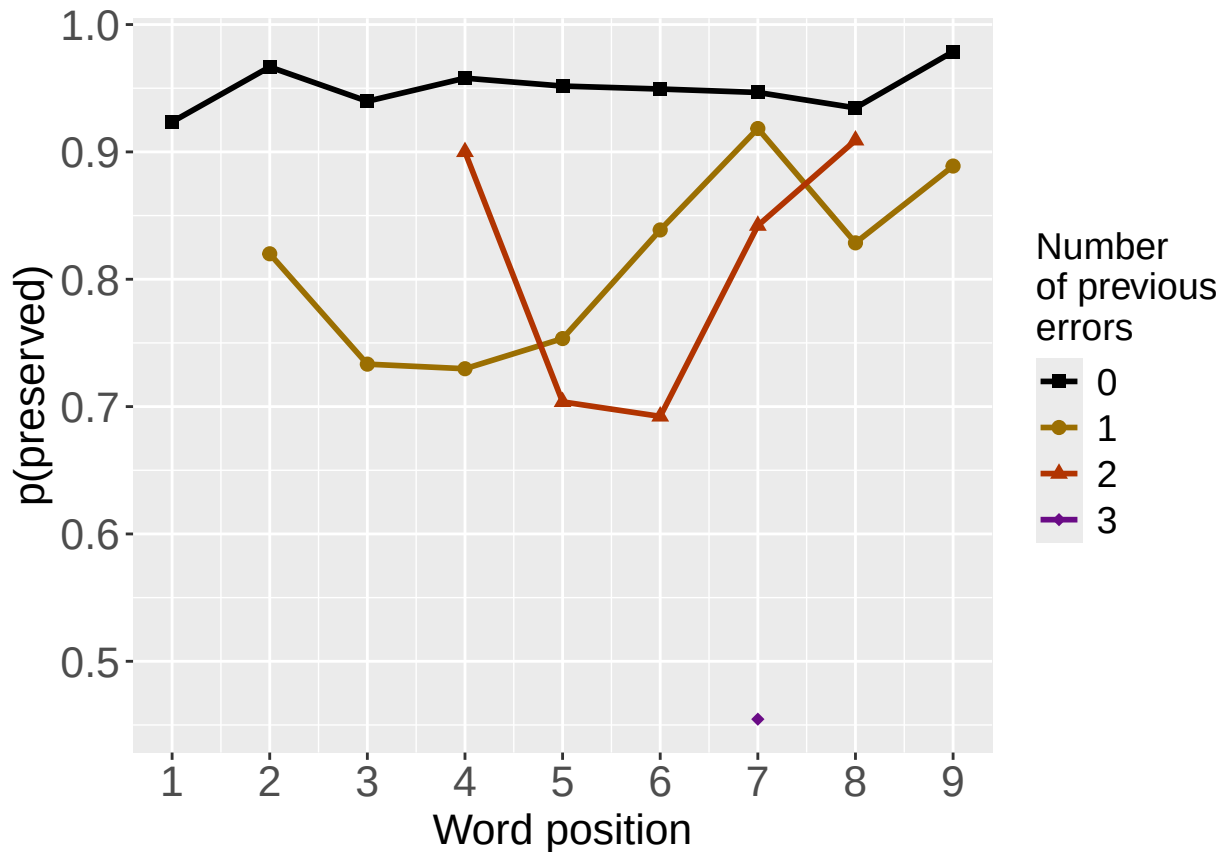
```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)
```



```
PrevErrPlot<-PlotObsPreviousError(PosDat,palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
```

```
print(PrevErrPlot)
```

```

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevCorPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevErrPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")

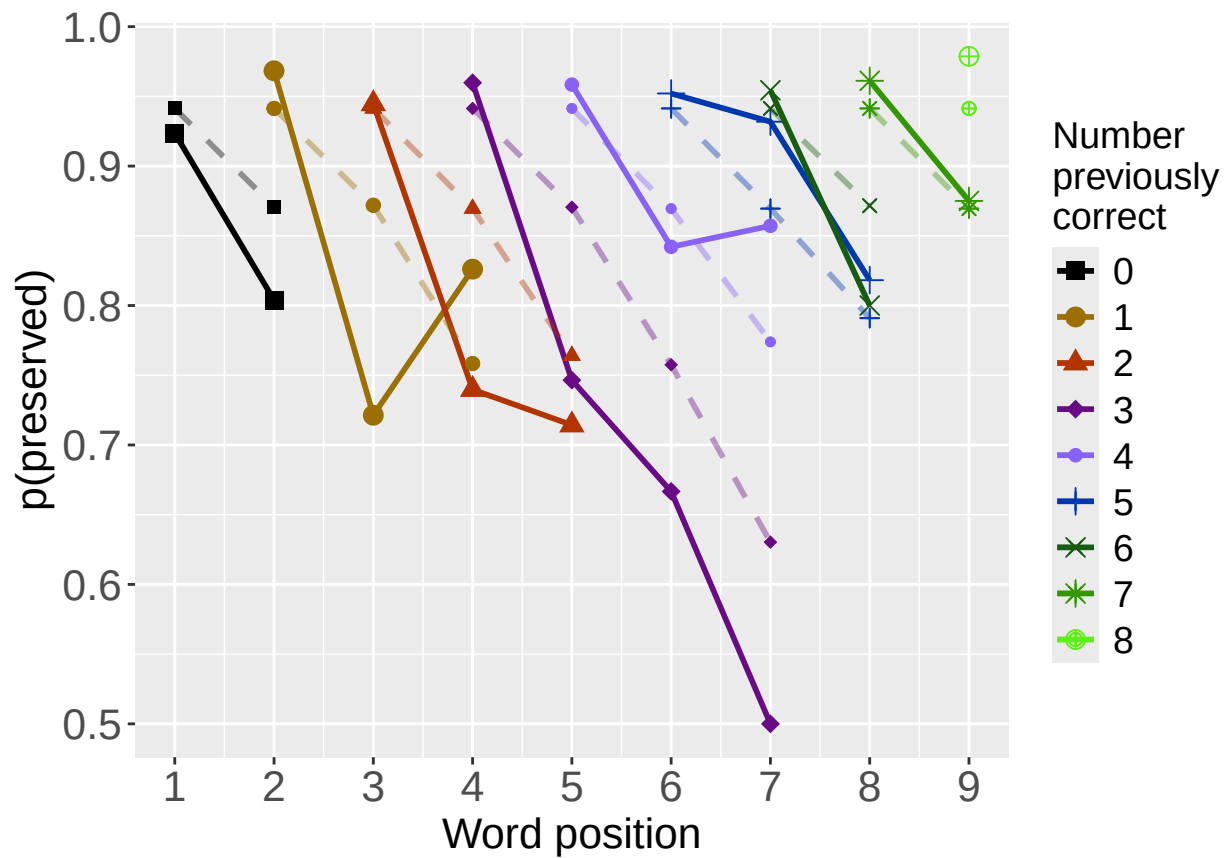
## Saving 6.5 x 4.5 in image

# plot prev err and prev cor with predicted values
MEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
PosDat$MEPred<-fitted(MEModel)

PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","MEPred",palette_values,shape_values)

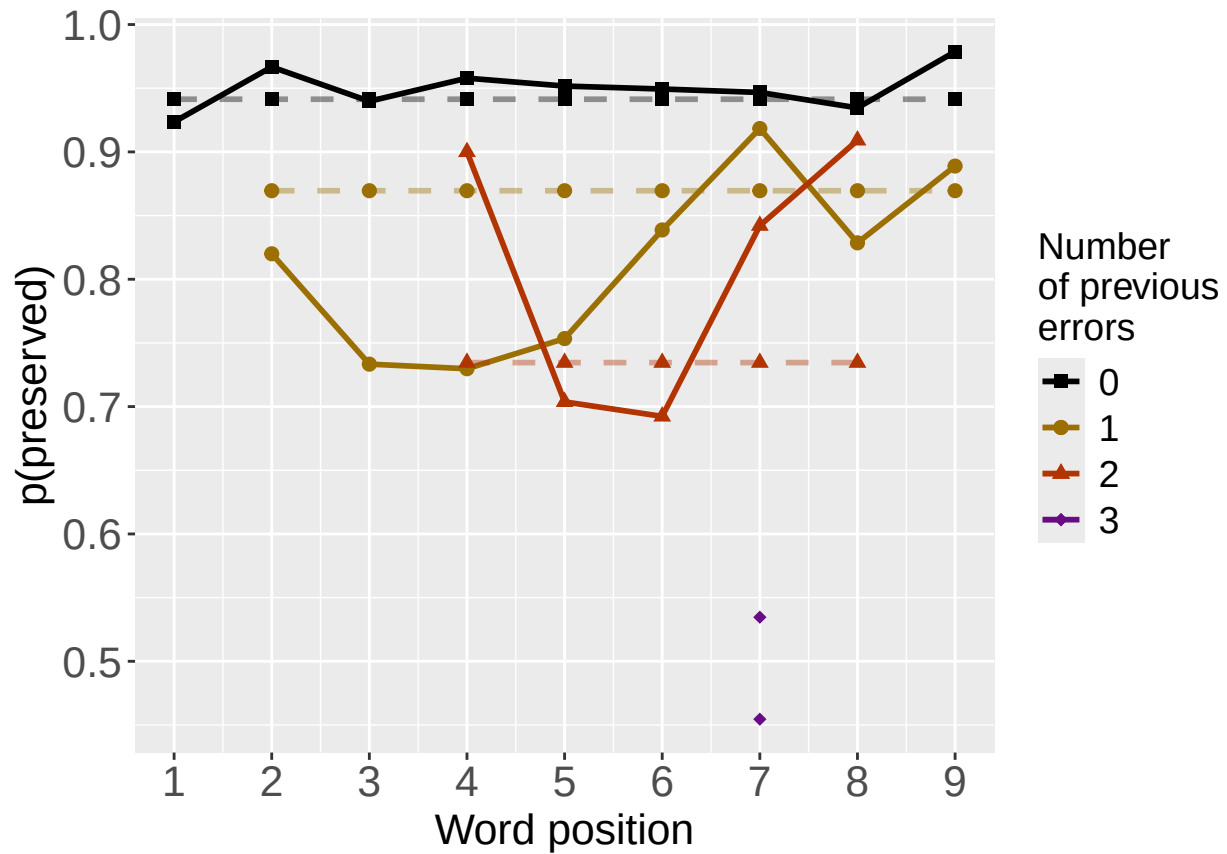
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)

```



```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","MEPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
print(PrevErrPlot)
```



```

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                  CurTask,"PrevCorPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                  CurTask,"PrevErrPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

```

```

CumAICSummary <- NULL

#####
# level 2 -- Add position squared (quadratic with position)
#####

# After establishing the primary variable, see about additions

BestMEFactor<-BestMEModelFormula
OtherFactor<-"I(pos^2) + pos"
OtherFactorName<-"quadratic_by_pos"

if(!grepl("pos",BestMEFactor,fixed = TRUE)){ # if best model does not include pos
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor,OtherFactorName)
  CumAICSummary <- AICSummary
  kable(AICSummary)
}

```

```

## *****
## model index: 1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.776      -0.879
##
## Degrees of Freedom: 4021 Total (i.e. Null);  4020 Residual
## Null Deviance:      2180
## Residual Deviance: 2052  AIC: 2056
## log likelihood:  -1025.771
## Nagelkerke R2:  0.07486923
## % pres/err predicted correctly:  -542.6556
## % of predictable range [ (model-null)/(1-null) ]:  0.04708872
## *****
## model index: 2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",

```

```

##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      I(pos^2)      pos
##      2.704902      -0.940757      0.008194      -0.018433
##
## Degrees of Freedom: 4021 Total (i.e. Null);  4018 Residual
## Null Deviance:      2180
## Residual Deviance: 2048  AIC: 2056
## log likelihood:  -1024.074
## Nagelkerke R2:  0.07682226
## % pres/err predicted correctly:  -541.871
## % of predictable range [ (model-null)/(1-null) ]:  0.04846388
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)      pos
##      2.770121      0.002423      -0.083995
##
## Degrees of Freedom: 4021 Total (i.e. Null);  4019 Residual
## Null Deviance:      2180
## Residual Deviance: 2174  AIC: 2180
## log likelihood:  -1087.115
## Nagelkerke R2:  0.003150882
## % pres/err predicted correctly:  -568.7586
## % of predictable range [ (model-null)/(1-null) ]:  0.001335752
## *****

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos
preserved ~ CumErr	2055.542	0.000000	1.0000000	0.575182	0.0748692	2.775734	-0.8790237	NA	NA
preserved ~ CumErr + I(pos^2) + pos	2056.148	0.606051	0.7385803	0.424818	0.0768223	2.704902	-0.9407565	0.0081940	-0.0184332
preserved ~ I(pos^2) + pos	2180.230	124.688065	0.0000000	0.000000	0.0031509	2.770121	NA	0.0024234	-0.0839953

```
#####
# level 2 -- Add length
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"stimlen"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}

## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumErr
## 2.776 -0.879
##
## Degrees of Freedom: 4021 Total (i.e. Null); 4020 Residual
## Null Deviance: 2180
## Residual Deviance: 2052 AIC: 2056
## log likelihood: -1025.771
## Nagelkerke R2: 0.07486923
## % pres/err predicted correctly: -542.6556
## % of predictable range [ (model-null)/(1-null) ]: 0.04708872
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumErr stimlen
## 2.54648 -0.89027 0.03023
##
## Degrees of Freedom: 4021 Total (i.e. Null); 4019 Residual
## Null Deviance: 2180
## Residual Deviance: 2051 AIC: 2057
## log likelihood: -1025.44
## Nagelkerke R2: 0.07525031
## % pres/err predicted correctly: -542.4047
## % of predictable range [ (model-null)/(1-null) ]: 0.04752844
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
```

```
## (Intercept)      stimlen
##      2.71570      -0.02971
##
## Degrees of Freedom: 4021 Total (i.e. Null);  4020 Residual
## Null Deviance:      2180
## Residual Deviance: 2179  AIC: 2183
## log likelihood:  -1089.421
## Nagelkerke R2:   0.0004123246
## % pres/err predicted correctly:  -569.4232
## % of predictable range [ (model-null)/(1-null) ]:  0.0001708448
## *****
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	stimlen
preserved ~ CumErr	2055.542	0.0000	1.0000000	0.6612792	0.0748692	2.775734	- 0.8790237	NA
preserved ~ CumErr + stimlen	2056.880	1.3380	0.5122205	0.3387208	0.0752503	2.546483	- 0.8902670	0.0302318
preserved ~ stimlen	2182.841	127.2995	0.0000000	0.0000000	0.0004123	2.715702	NA	- 0.0297103

```
#####
# level 2 -- add cumulative preserved
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"CumPres"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```

```
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres
##      2.57160      -0.88380      0.08234
##
## Degrees of Freedom: 4021 Total (i.e. Null);  4019 Residual
## Null Deviance:      2180
## Residual Deviance: 2045  AIC: 2051
## log likelihood:  -1022.304
## Nagelkerke R2:   0.07885756
## % pres/err predicted correctly:  -541.5095
## % of predictable range [ (model-null)/(1-null) ]:  0.04909748
## *****
## model index:  1
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.776      -0.879
##
## Degrees of Freedom: 4021 Total (i.e. Null);  4020 Residual
## Null Deviance:      2180
## Residual Deviance: 2052 AIC: 2056
## log likelihood: -1025.771
## Nagelkerke R2:  0.07486923
## % pres/err predicted correctly: -542.6556
## % of predictable range [ (model-null)/(1-null) ]:  0.04708872
## *****
## model index:  3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      2.30348      0.07334
##
## Degrees of Freedom: 4021 Total (i.e. Null);  4020 Residual
## Null Deviance:      2180
## Residual Deviance: 2173 AIC: 2177
## log likelihood: -1086.709
## Nagelkerke R2:  0.003632283
## % pres/err predicted correctly: -568.6887
## % of predictable range [ (model-null)/(1-null) ]:  0.00145824
## *****
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	CumPres
preserved ~ CumErr + CumPres	2050.608	0.000000	1.000000	0.9217923	0.0788576	2.571599	- 0.8838005	0.0823401
preserved ~ CumErr	2055.542	4.933904	0.084843	0.0782077	0.0748692	2.775734	- 0.8790237	NA
preserved ~ CumPres	2177.419	126.810792	0.000000	0.0000000	0.0036323	2.303477	NA	0.0733389

```
#####
# level 2 -- Add linear position (NOT quadratic)
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"pos"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```



```

## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##         data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      pos
##      2.58759      -0.93858      0.05303
##
## Degrees of Freedom: 4021 Total (i.e. Null);  4019 Residual
## Null Deviance:      2180
## Residual Deviance: 2049  AIC: 2055
## log likelihood:  -1024.287
## Nagelkerke R2:  0.07657698
## % pres/err predicted correctly:  -542.047
## % of predictable range [ (model-null)/(1-null) ]:  0.04815534
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##         data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.776      -0.879
##
## Degrees of Freedom: 4021 Total (i.e. Null);  4020 Residual
## Null Deviance:      2180
## Residual Deviance: 2052  AIC: 2056
## log likelihood:  -1025.771
## Nagelkerke R2:  0.07486923
## % pres/err predicted correctly:  -542.6556
## % of predictable range [ (model-null)/(1-null) ]:  0.04708872
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##         data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##      2.73419      -0.06247
##
## Degrees of Freedom: 4021 Total (i.e. Null);  4020 Residual
## Null Deviance:      2180
## Residual Deviance: 2174  AIC: 2178
## log likelihood:  -1087.135
## Nagelkerke R2:  0.003126797
## % pres/err predicted correctly:  -568.7646
## % of predictable range [ (model-null)/(1-null) ]:  0.001325101
## *****

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	pos
preserved ~ CumErr + pos	2054.574	0.0000000	1.0000000	0.6186381	0.0765770	2.587591	- 0.9385786	0.0530319
preserved ~ CumErr	2055.542	0.9675433	0.6164539	0.3813619	0.0748692	2.775734	- 0.8790237	NA
preserved ~ pos	2178.271	123.6961888	0.0000000	0.0000000	0.0031268	2.734188	NA	- 0.0624749

```
CumAICSummary <- CumAICSummary %>% arrange(AIC)
BestModelFormulaL2 <- CumAICSummary$Model[BestModelIndexL2]
write.csv(CumAICSummary, paste0(TablesDir, CurPat, "_",
                                RunLabel, "_", CurTask,
                                "_main_effects_plus_one_model_summary.csv"), row.names = FALSE)
kable(CumAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos	stimlen	CumPres
preserved ~ CumErr + CumPres	2050.608	0.0000000	1.0000000	0.9217923	0.0788525	2.571599	- 0.8838005	NA	NA	NA	0.0823401
preserved ~ CumErr + pos	2054.574	0.0000000	1.0000000	0.6186381	0.0765770	2.587591	- 0.9385786	NA	0.0530319	NA	NA
preserved ~ CumErr	2055.542	0.0000000	1.0000000	0.5751820	0.0748692	2.775734	- 0.8790237	NA	NA	NA	NA
preserved ~ CumErr	2055.542	0.0000000	1.0000000	0.6612792	0.0748692	2.775734	- 0.8790237	NA	NA	NA	NA
preserved ~ CumErr	2055.542	0.9339045	0.0848430	0.0782077	0.0748692	2.775734	- 0.8790237	NA	NA	NA	NA
preserved ~ CumErr	2055.542	0.9675433	0.6164539	0.3813619	0.0748692	2.775734	- 0.8790237	NA	NA	NA	NA
preserved ~ CumErr + I(pos^2) + pos	2056.148	6.6060510	0.7385803	0.4248180	0.0768223	2.704902	- 0.9407565	0.0081940	- 0.0184332	NA	NA
preserved ~ CumErr + stimlen	2056.880	0.3380002	0.5122205	0.3387208	0.0752503	2.546483	- 0.8902670	NA	NA	0.0302318	NA
preserved ~ CumPres	2177.419	26.8107920	0.0000000	0.0000000	0.0036323	2.303477	NA	NA	NA	NA	0.0733389
preserved ~ pos	2178.271	123.6961888	0.0000000	0.0000000	0.0031268	2.734188	NA	NA	- 0.0624749	NA	NA
preserved ~ I(pos^2) + pos	2180.230	24.6880640	0.0000000	0.0000000	0.0031502	2.770121	NA	0.0024234	- 0.0839953	NA	NA
preserved ~ stimlen	2182.841	27.2995187	0.0000000	0.0000000	0.0004123	2.715702	NA	NA	NA	- 0.0297103	NA

```
# explore influence of frequency and length

if(grepl("stimlen", BestModelFormulaL2)){ # if length is already in the formula
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2, " + log_freq")
  )
}
```

```

}else if(grepl("log_freq",BestModelFormulaL2)){ # if log_freq is already in the formula
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2," + stimlen")
  )
}else{
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2," + log_freq"),
    paste0(BestModelFormulaL2," + stimlen"),
    paste0(BestModelFormulaL2," + stimlen + log_freq")
  )
}
}

Level3Res<-TestModels(Level3ModelEquations,PosDat)

```

```

## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres      log_freq
##      2.55580      -0.85850      0.09451      0.11227
##
## Degrees of Freedom: 4021 Total (i.e. Null); 4018 Residual
## Null Deviance:      2180
## Residual Deviance: 2032 AIC: 2040
## log likelihood: -1016.233
## Nagelkerke R2: 0.08582456
## % pres/err predicted correctly: -540.8635
## % of predictable range [ (model-null)/(1-null) ]: 0.05022983
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres      stimlen      log_freq
##      2.33159      -0.86784      0.08591      0.03245      0.11935
##
## Degrees of Freedom: 4021 Total (i.e. Null); 4017 Residual
## Null Deviance:      2180
## Residual Deviance: 2032 AIC: 2042
## log likelihood: -1015.924
## Nagelkerke R2: 0.08617866
## % pres/err predicted correctly: -540.685
## % of predictable range [ (model-null)/(1-null) ]: 0.05054263
## *****
## model index: 1

```

```
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres
##      2.57160      -0.88380      0.08234
##
## Degrees of Freedom: 4021 Total (i.e. Null);  4019 Residual
## Null Deviance:      2180
## Residual Deviance: 2045 AIC: 2051
## log likelihood: -1022.304
## Nagelkerke R2:  0.07885756
## % pres/err predicted correctly: -541.5095
## % of predictable range [ (model-null)/(1-null) ]:  0.04909748
## *****
## model index:  4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres      stimlen
##      2.617451      -0.881473      0.084325      -0.006693
##
## Degrees of Freedom: 4021 Total (i.e. Null);  4018 Residual
## Null Deviance:      2180
## Residual Deviance: 2045 AIC: 2053
## log likelihood: -1022.29
## Nagelkerke R2:  0.07887405
## % pres/err predicted correctly: -541.5338
## % of predictable range [ (model-null)/(1-null) ]:  0.04905493
## *****
## model index:  2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
##      2.486
##
## Degrees of Freedom: 4021 Total (i.e. Null);  4021 Residual
## Null Deviance:      2180
## Residual Deviance: 2180 AIC: 2182
## log likelihood: -1089.768
## Nagelkerke R2:  2.653751e-16
## % pres/err predicted correctly: -569.5206
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****
```

```
BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[[BestModelIndexL3]]
```

```

AICSummary<-data.frame(Model=Level3Res$Model,
                        AIC=Level3Res$AIC,
                        row.names = Level3Res$Model)
AICSummary$DeltaAIC<-AICSummary$AIC-AICSummary$AIC[1]
AICSummary$AICexp<-exp(-0.5*AICSummary$DeltaAIC)
AICSummary$AICwt<-AICSummary$AICexp/sum(AICSummary$AICexp)
AICSummary$NagR2<-Level3Res$NagR2

AICSummary <- merge(AICSummary,Level3Res$CoefficientValues,
                    by='row.names',sort=FALSE)
AICSummary <- subset(AICSummary, select = -c(Row.names))

write.csv(AICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                           CurTask,"_main_effects_plus_one_len_freq_summary.csv"),
          row.names = FALSE)

kable(AICSummary)

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	CumPres	log_freq	stimlen
preserved ~ CumErr + CumPres + log_freq	2040.467	0.0000001	0.000000	0.662379	0.085824	5555805	-	0.09451	0.1122734	NA
preserved ~ CumErr + CumPres + stimlen + log_freq	2041.849	1.381947	0.501088	0.331910	0.086178	7331593	-	0.085913	0.119354	0.0324495
preserved ~ CumErr + CumPres	2050.608	0.141230	0.006278	0.600415	0.078857	5571599	-	0.082340	1NA	NA
preserved ~ CumErr + CumPres + stimlen	2052.579	2.112530	0.002343	0.001552	0.078874	41617451	-	0.084324	8NA	-
preserved ~ 1	2181.535	141.06850	0.000000	0.000000	0.000000	0486254	NA	NA	NA	NA

```

# BestModelIndex is set by the parameter file and is used to choose
# a model that is essentially equivalent to the lowest AIC model, but
# simpler (and with a slightly higher AIC value, so not in first position)
BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[BestModelIndexL3]

```

```

BestModel<-BestModelL3
BestModelDeletions<-arrange(dropterm(BestModel),desc(AIC))
# order by terms that increase AIC the most when they are the one dropped
BestModelDeletions

```

```

## Single term deletions
##
## Model:
## preserved ~ CumErr + CumPres + log_freq
##      Df Deviance   AIC
## CumErr    1   2153.8 2159.8
## log_freq   1   2044.6 2050.6
## CumPres    1   2041.5 2047.5
## <none>      0   2032.5 2040.5

```

```

#####
# Single deletions from best model

```

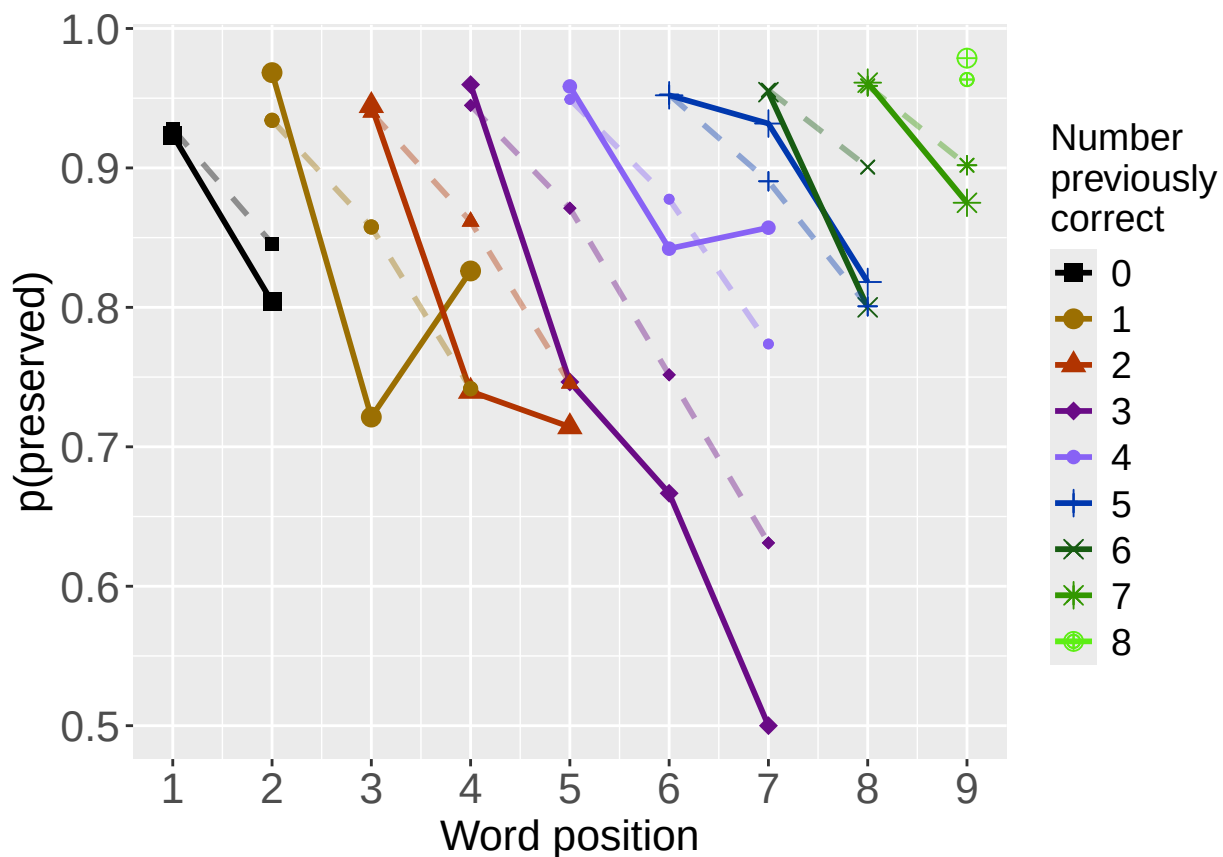
```
#####

write.csv(BestModelDeletions,paste0(TablesDir,CurPat,"_",
                                   RunLabel,"_",CurTask,
                                   "_best_model_single_term_deletions.csv"),
          row.names = TRUE)

# plot prev err and prev cor with predicted values
PosDat$OAPred<-fitted(BestModel)

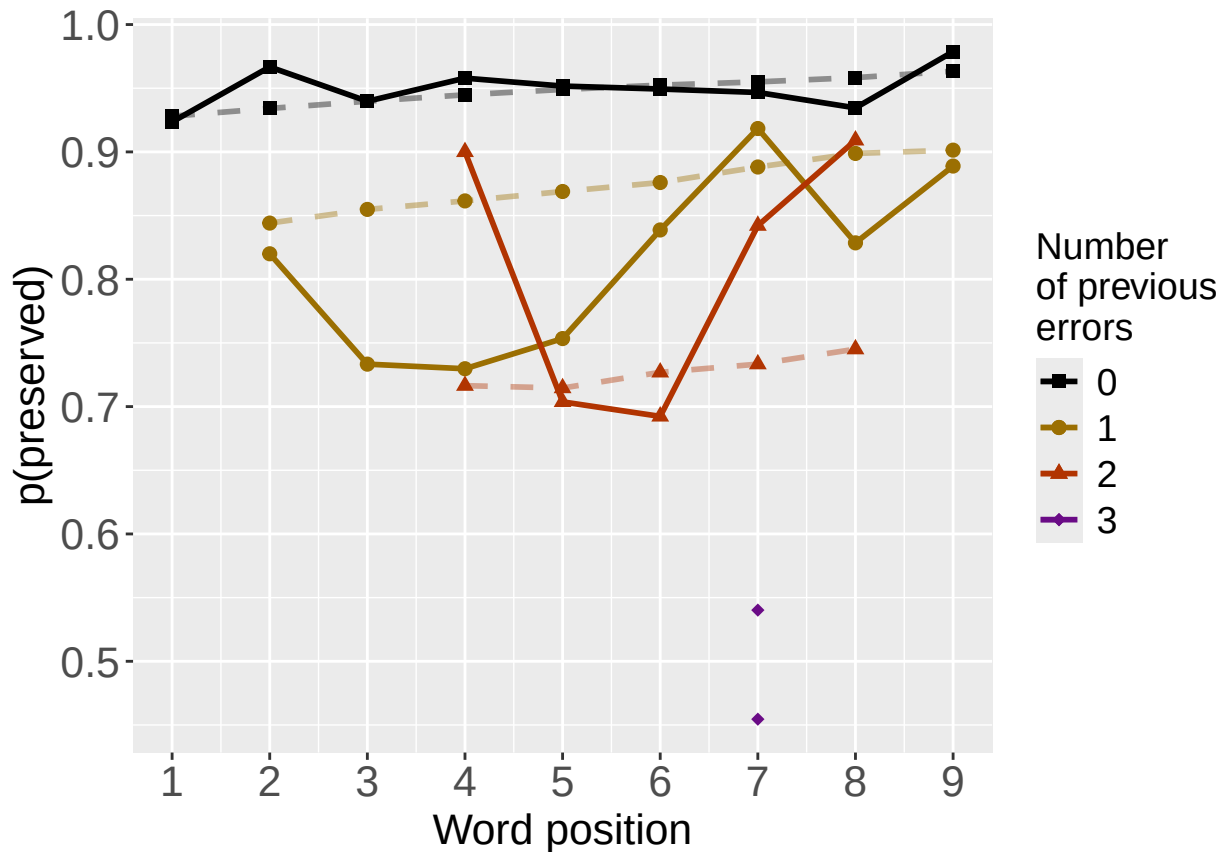
PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","OAPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)
```



```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","OAPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
print(PrevErrPlot)
```



```

PlotName<-paste0(FigDir,CurPat,"_",
                 RunLabel,"_",CurTask,
                 "_ObsPred_BestFullModel")
ggsave(filename=paste(PlotName,"_prev_correct.tif",sep=""),plot=PrevCorPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image
ggsave(filename=paste(PlotName,"_prev_error.tif",sep=""),plot=PrevErrPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image
if(DoSimulations){
  BestModelFormulaL3Rnd <- BestModelFormulaL3
  if(grepl("CumPres",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumPres","RndCumPres",BestModelFormulaL3Rnd)
  }
  if(grepl("CumErr",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumErr","RndCumErr",BestModelFormulaL3Rnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestModelFormulaL3Rnd),
                       family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
  }
}

```

```

}
ModelNames<-c(paste0("***",BestModelFormulaL3),
              rep(BestModelFormulaL3Rnd,RandomSamples))
AICValues <- c(BestModelL3$aic,RndModelAIC)
BestModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
BestModelRndDF <- BestModelRndDF %>% arrange(AIC)
BestModelRndDF <- rbind(BestModelRndDF,
                        data.frame(Name=c("Random average"),
                                   AIC=c(mean(RndModelAIC))))
BestModelRndDF <- rbind(BestModelRndDF,
                        data.frame(Name=c("Random SD"),
                                   AIC=c(sd(RndModelAIC))))
write.csv(BestModelRndDF,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                                "_best_model_with_random_cum_term.csv"),
          row.names = FALSE)
}

# plot influence factors
num_models <- nrow(BestModelDeletions) - 1
# minus 1 because last
# model is always <none> and we don't want to include that

if(num_models > 4){
  last_model_num <- 4
}else{
  last_model_num <- num_models
}

FinalModelSet<-ModelSetFromDeletions(BestModelFormulaL3,
                                     BestModelDeletions,
                                     last_model_num)

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_FactorPlots")

FactorPlot<-PlotModelSetWithFreq(PosDat,FinalModelSet,
                                 palette_values,FinalModelSet,PptID=CurPat)

## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.776      -0.879
##
## Degrees of Freedom: 4021 Total (i.e. Null); 4020 Residual
## Null Deviance:      2180
## Residual Deviance: 2052 AIC: 2056
## log likelihood: -1025.771
## Nagelkerke R2: 0.07486923
## % pres/err predicted correctly: -542.6556
## % of predictable range [ (model-null)/(1-null) ]: 0.04708872

```



```

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      log_freq
##      2.7877      -0.8544      0.1014
##
## Degrees of Freedom: 4021 Total (i.e. Null); 4019 Residual
## Null Deviance:      2180
## Residual Deviance: 2042 AIC: 2048
## log likelihood: -1020.753
## Nagelkerke R2: 0.08063921
## % pres/err predicted correctly: -542.1796
## % of predictable range [ (model-null)/(1-null) ]: 0.04792291
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      log_freq      CumPres
##      2.55580      -0.85850      0.11227      0.09451
##
## Degrees of Freedom: 4021 Total (i.e. Null); 4018 Residual
## Null Deviance:      2180
## Residual Deviance: 2032 AIC: 2040
## log likelihood: -1016.233
## Nagelkerke R2: 0.08582456
## % pres/err predicted correctly: -540.8635
## % of predictable range [ (model-null)/(1-null) ]: 0.05022983
## *****

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.

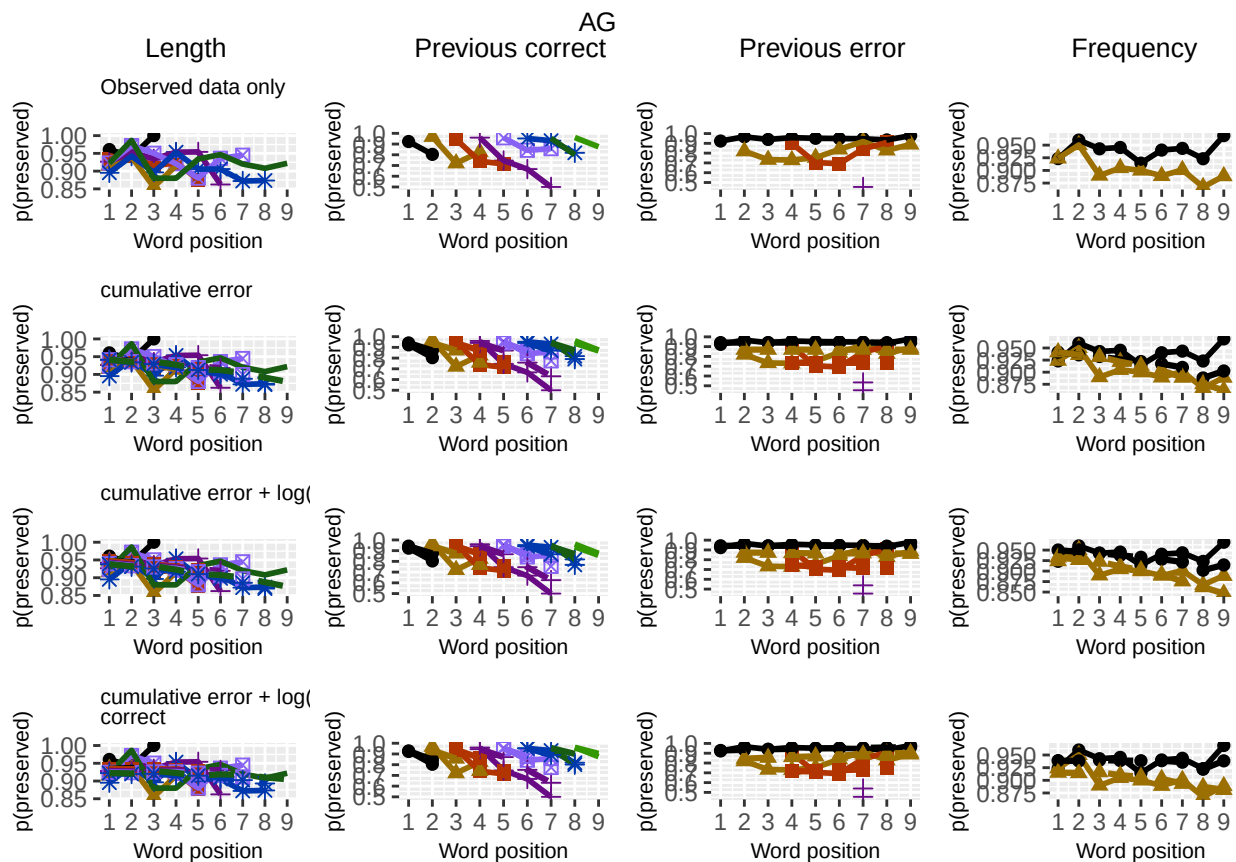
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

```

```

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 5 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 5 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 5 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 5 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 5 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 5 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 5 rows containing missing values or values outside the scale range (`geom_point()`).
ggsave(paste0(PlotName, ".tif"), plot=FactorPlot, width = 360, height=400, units="mm", device="tiff", compress=
# use \blandscape and \elandscape to make markdown plots landscape if needed
FactorPlot

```



```
DA.Result<-dominanceAnalysis(BestModel)
DAContributionAverage<-ConvertDominanceResultToTable(DA.Result)
write.csv(DAContributionAverage,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                     CurTask,"_dominance_analysis_table.csv"),
          row.names = TRUE)
kable(DAContributionAverage)
```

	CumErr	CumPres	log_freq
McFadden	0.0571994	0.0034729	0.0068047
SquaredCorrelation	0.0304282	0.0018444	0.0036330
Nagelkerke	0.0304282	0.0018444	0.0036330
Estrella	0.0314984	0.0019157	0.0037364

	deviance	deviance_explained
CumErr + log_freq + CumPres	2032.467	147.0685
CumErr + log_freq	2041.507	138.0286
CumErr	2051.542	127.9934
null	2179.535	0.0000

	percent_explained	percent_of_explained_deviance	increment_in_explained
CumErr + log_freq + CumPres	6.747700	100.00000	6.146709
CumErr + log_freq	6.332939	93.85329	6.823530
CumErr	5.872507	87.02976	87.029761
null	0.000000	NA	0.000000

```
deviance_differences_df<-GetDevianceSet(FinalModelSet,PosDat)
write.csv(deviance_differences_df,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                         CurTask,"_deviance_differences_table.csv"),
          row.names = FALSE)
deviance_differences_df
```

```
##                                model deviance deviance_explained percent_explained percent_of_explained_deviance
## CumErr + log_freq + CumPres CumErr + log_freq + CumPres 2032.467          147.0685          6.747700          100.00000
## CumErr + log_freq          CumErr + log_freq 2041.507          138.0286          6.332939          93.85329
## CumErr                    CumErr 2051.542          127.9934          5.872507          87.02976
## null                      null 2179.535          0.0000          0.000000          NA
##                                increment_in_explained
## CumErr + log_freq + CumPres          6.146709
## CumErr + log_freq          6.823530
## CumErr          87.029761
## null          0.000000
```

```
kable(deviance_differences_df[,2:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
kable(deviance_differences_df[,c(4:6)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
NagContributions <- t(DAContributionAverage[3,])
NagTotal<-sum(NagContributions)
```

```

NagPercents <- NagContributions / NagTotal
NagResultTable <- data.frame(NagContributions = NagContributions, NagPercents = NagPercents)
names(NagResultTable)<-c("NagContributions","NagPercents")
write.csv(NagResultTable,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                                "_nagelkerke_contributions_table.csv"),row.names = TRUE)
NagPercents

```

```

##           Nagelkerke
## CumErr    0.84744987
## CumPres   0.05136937
## log_freq  0.10118075

```

```

sse_results_list<-compare_SS_accounted_for(FinalModelSet,"preserved ~ 1",PosDat,N_cutoff=10)

```

```

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

```

model	p_accounted_for	model_deviance
preserved ~ CumErr+log_freq	0.8718837	2041.507
preserved ~ CumErr	0.8739472	2051.542
preserved ~ CumErr+log_freq+CumPres	0.8788894	2032.467

model	diff_CumErr+log_freq	diff_CumErr	diff_CumErr+log_freq+CumPres
preserved ~ CumErr+log_freq	0.0000000	-0.0020635	-0.0070057
preserved ~ CumErr	0.0020635	0.0000000	-0.0049421
preserved ~ CumErr+log_freq+CumPres	0.0070057	0.0049421	0.0000000

```
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
sse_table <- sse_results_table(sse_results_list)
write.csv(sse_table, paste0(TablesDir, CurPat, "_", RunLabel, "_",
                           CurTask, "_sse_results_table.csv"), row.names = TRUE)

sse_table
```

```
##              model p_accounted_for model_deviance diff_CumErr+log_freq diff_CumErr diff_CumErr+log_freq+CumPres
## 1 preserved ~ CumErr+log_freq      0.8718837      2041.507      0.0000000000 -0.002063522      -0.007005657
## 2 preserved ~ CumErr      0.8739472      2051.542      0.002063522      0.0000000000      -0.004942136
## 3 preserved ~ CumErr+log_freq+CumPres      0.8788894      2032.467      0.007005657      0.004942136      0.000000000
```

```
kable(sse_table[,1:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
write.csv(results_report_DF, paste0(TablesDir, CurPat, "_", RunLabel, "_",
                                   CurTask, "_results_report_df.csv"), row.names = FALSE)
```

```
ncols_in_table <- ncol(sse_table)
kable(sse_table[,c(1,4:ncols_in_table)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```